



Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

Machine Learning & Deep Learning

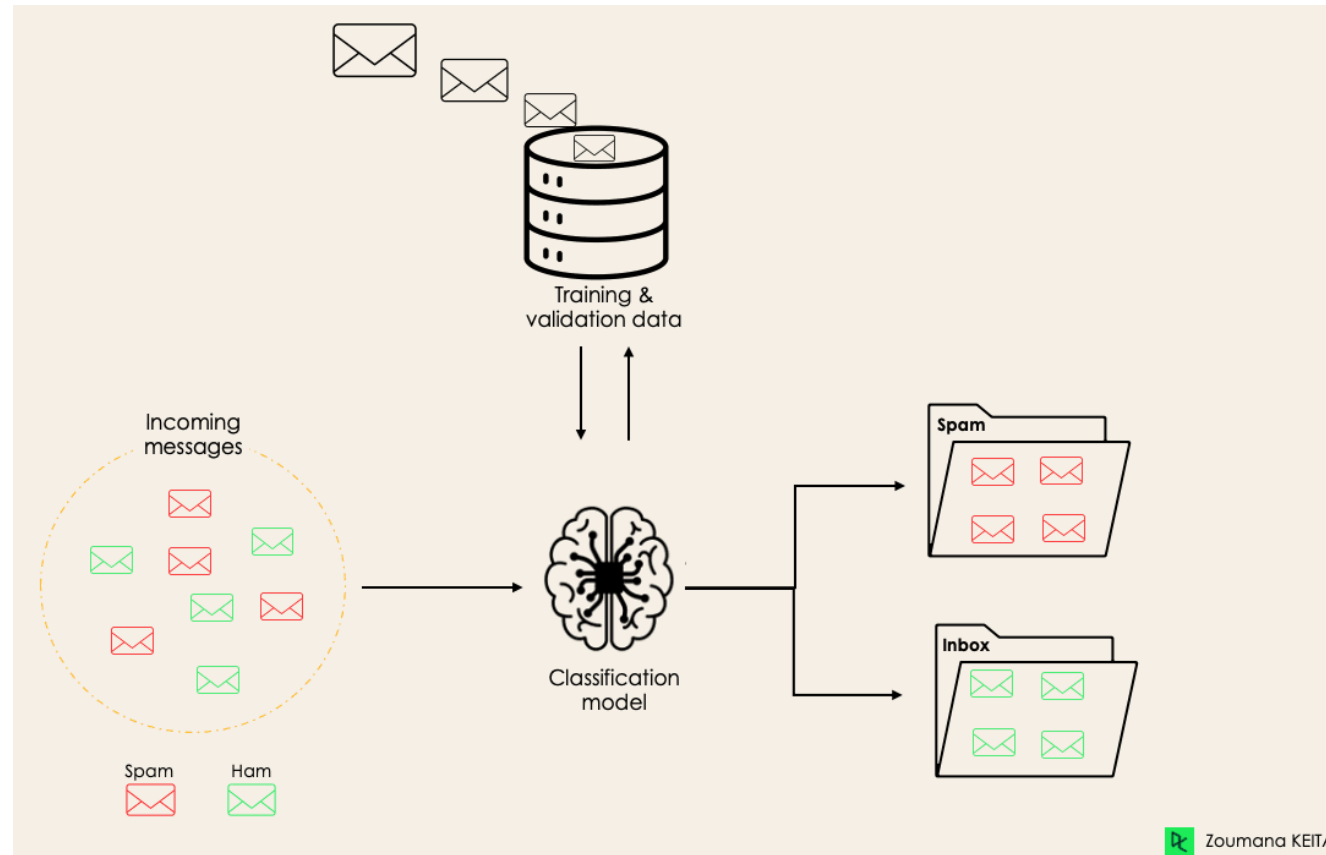
02: CLASSIFICATION



Motivation

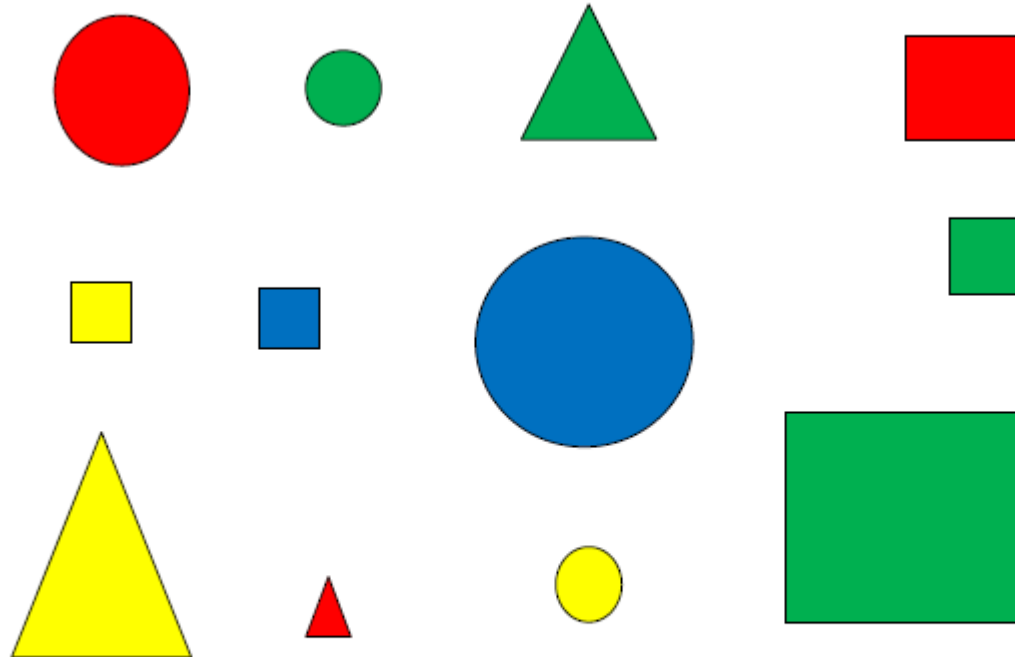
We now want the computer to make autonomous decisions (we focus on „classification“)

- So we address the question: „How to teach your computer to make intelligent decisions autonomously?“



Source & More Information: <https://www.datacamp.com/blog/classification-machine-learning>

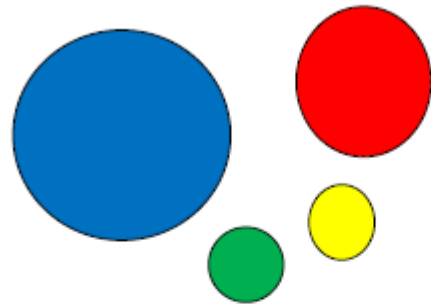
Defining classification



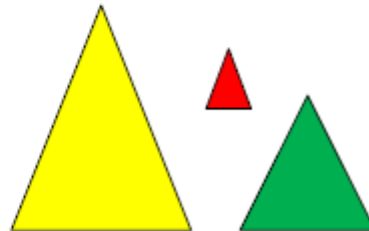
Classification:
„Clasify“ an instance as any of N pre-defined classes

Defining classification

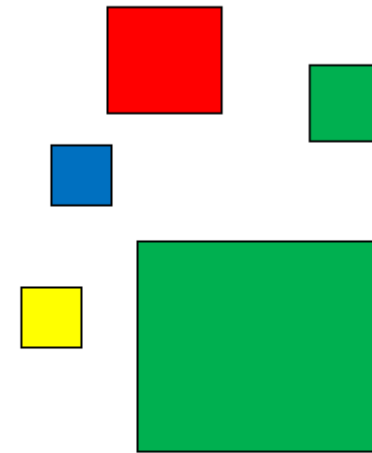
Example 1: Classify the instances as one of the three classes “circle”, “square”, “triangle”



class: circle



class: triangle



class: square

Defining classification

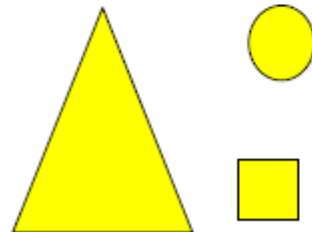
Example 2: Classify the instances as one of the four classes “blue”, “red”, “yellow”, “green”:



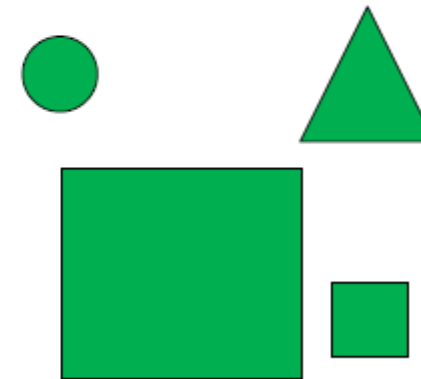
class: blue



class: red



class: yellow



class: green

Defining classification

- To solve the introduces classification examples, we could predefine the classes (e.g., circle, triangle, square) and predefine the features, e.g., a feature could be the number of edges:
circle: 0, triangle: 3, square: 4
- If this is possible, we should do so!
- **However, if we speak of “classification”, we typically have problem where predefining features and thresholds is not easily possible**
Hier Text eingeben
- E.g., the “Iris” dataset:



Iris setosa



Iris versicolor



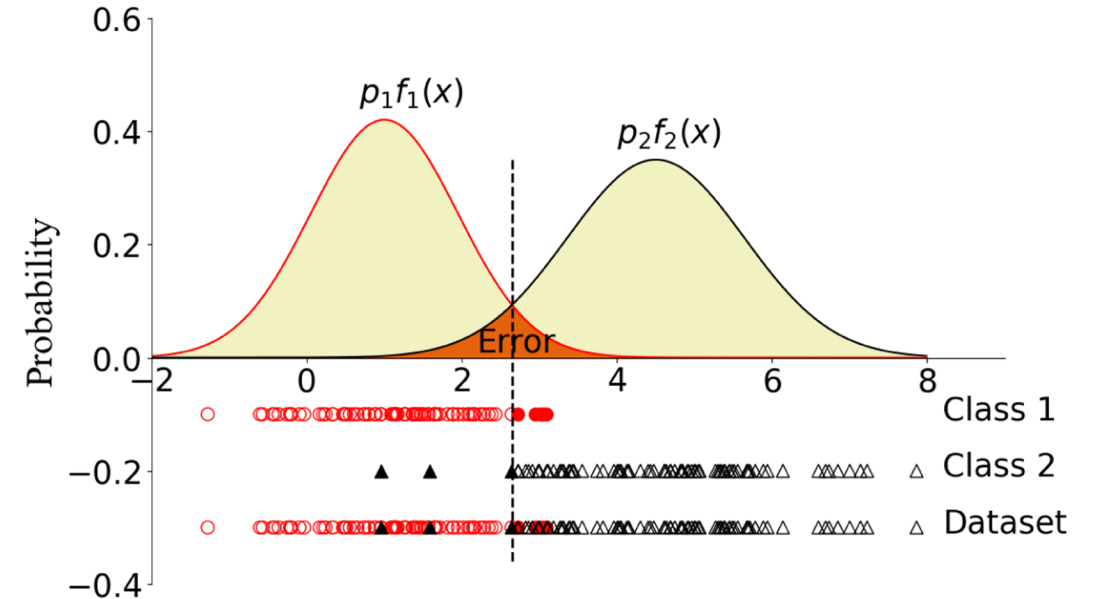
Iris virginica

We want to learn from data and try to find the relevant features and thresholds in the data.

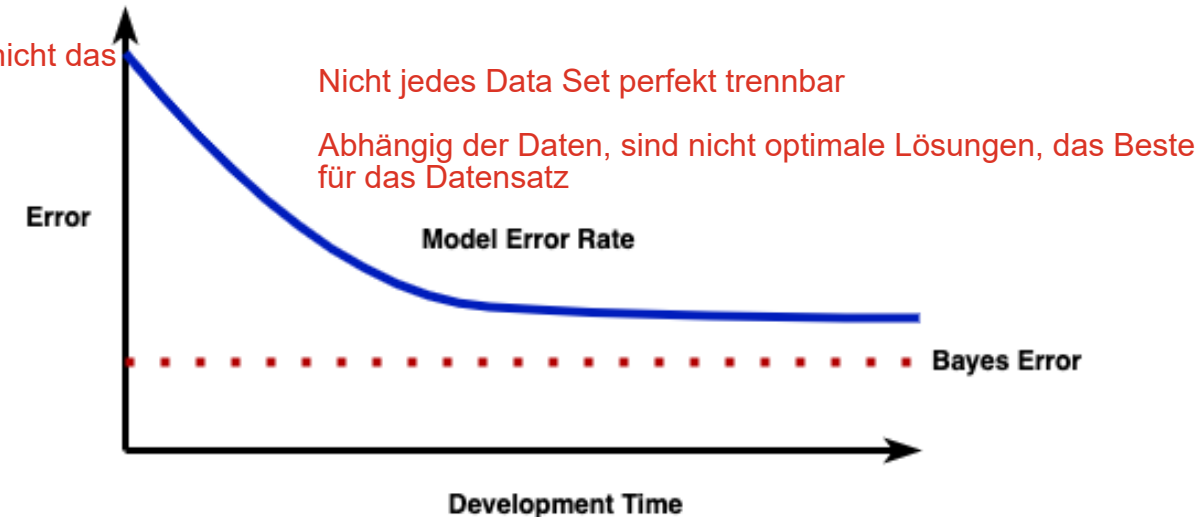
Statistical Background

Bayes Error Nicht-Unterscheidung

- The Bayes error lower bounds the error rate for any classifier
- The error is formulated independently of classifier properties
- The error exclusively depends on the properties of the data set in feature space F
- In applications, the Bayes error is usually unknown! *Einschätzung, dass es sehr komplexe Daten habe und deshalb nicht das Optimum für die Daten habe.*
- Calculating the Bayes error requires knowledge of:
 1. The type of probability density functions.
 2. The statistical parameters of the probability distributions.
 3. The prior class probabilities.

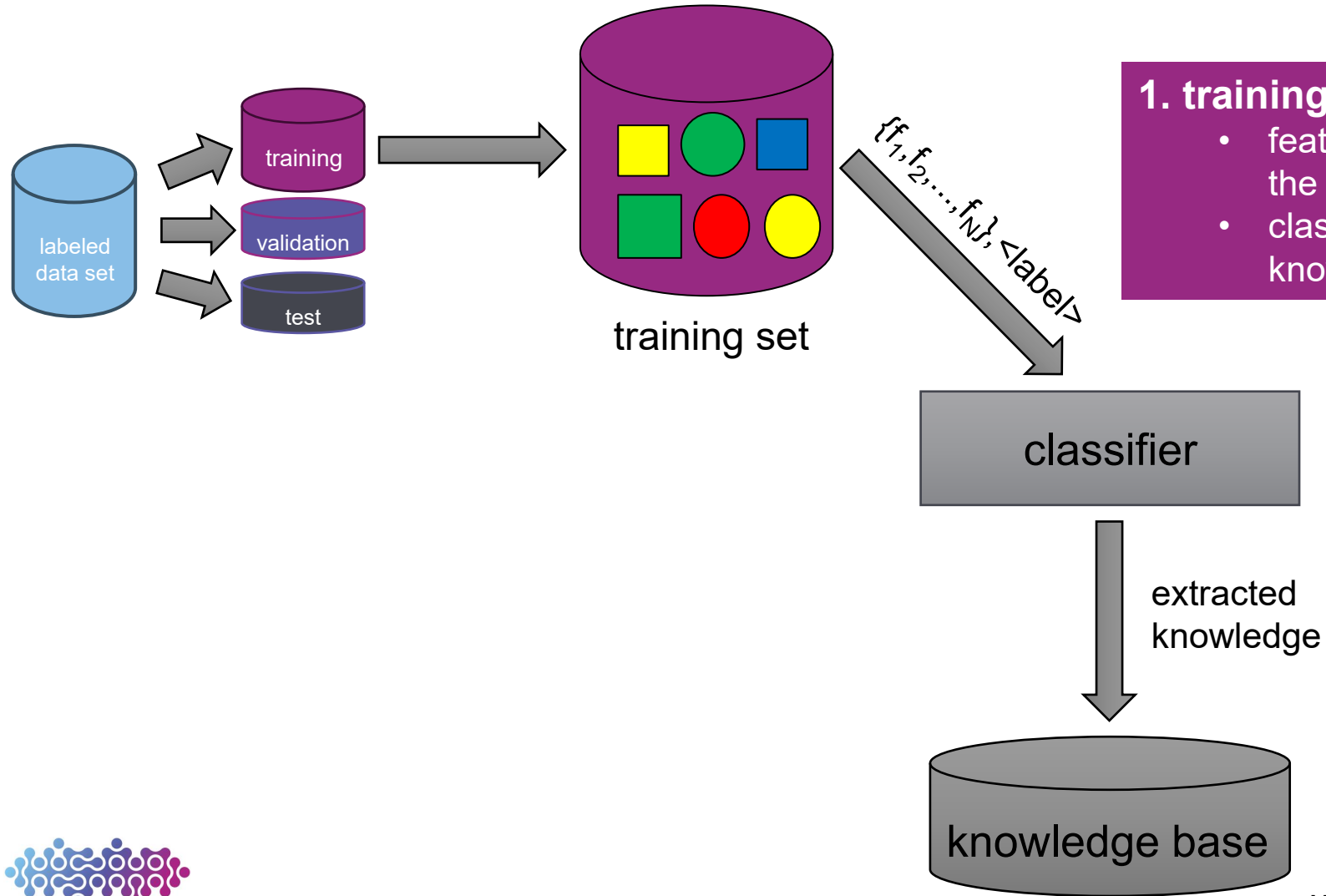


Source & More Information: Chen et al.



Source & More Information: <https://medium.com/data-science/what-is-bayes-error-4bfa1c9a0a0d> Dominik Hamm M.Sc.

Training Process



1. training:

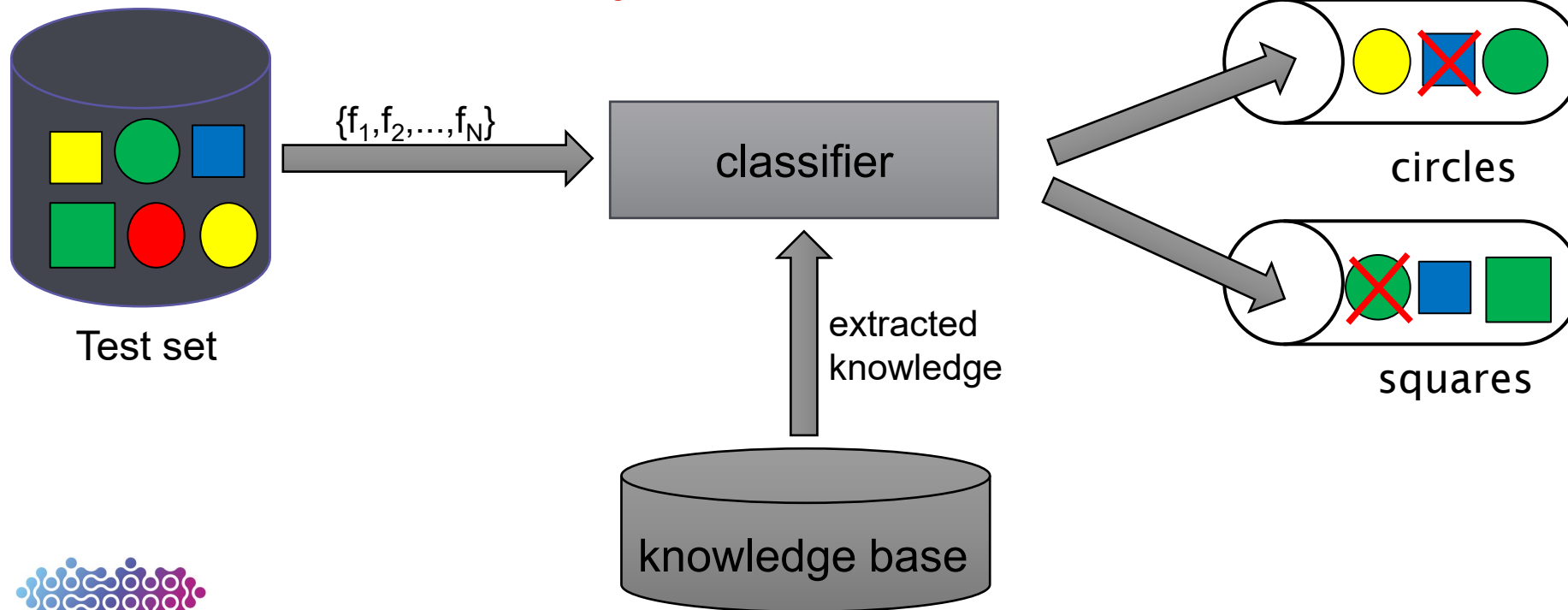
- feature vectors with class labels are passed to the classifier
- classifier extracts knowledge (type of knowledge depends on type of classifier)

Test Process

2. test:

- feature vectors without class labels are passed to the classifier (the labels are known though)
- classifier classifies the passed feature vectors
- accuracy is determined based on the classification and the known class labels

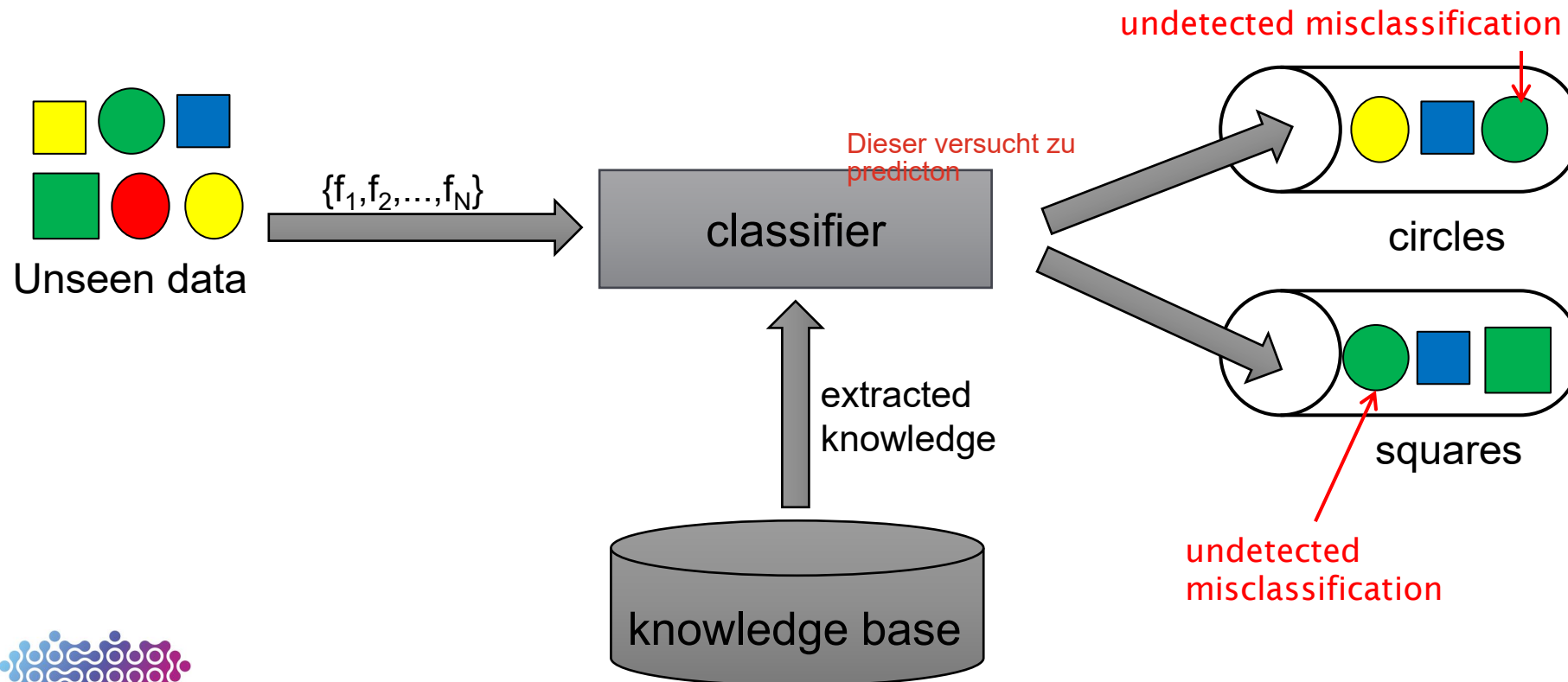
Nur mit Features, ohne dass Labels mitgegeben werden.



Operation Test Process

3. operation mode:

- classifier classifies unseen data based on the previously extracted knowledge
- in this stage the true labels are unknown!
- misclassifications are not detected



Process for Data Mining / Machine Learning (Evaluation)

Classification

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

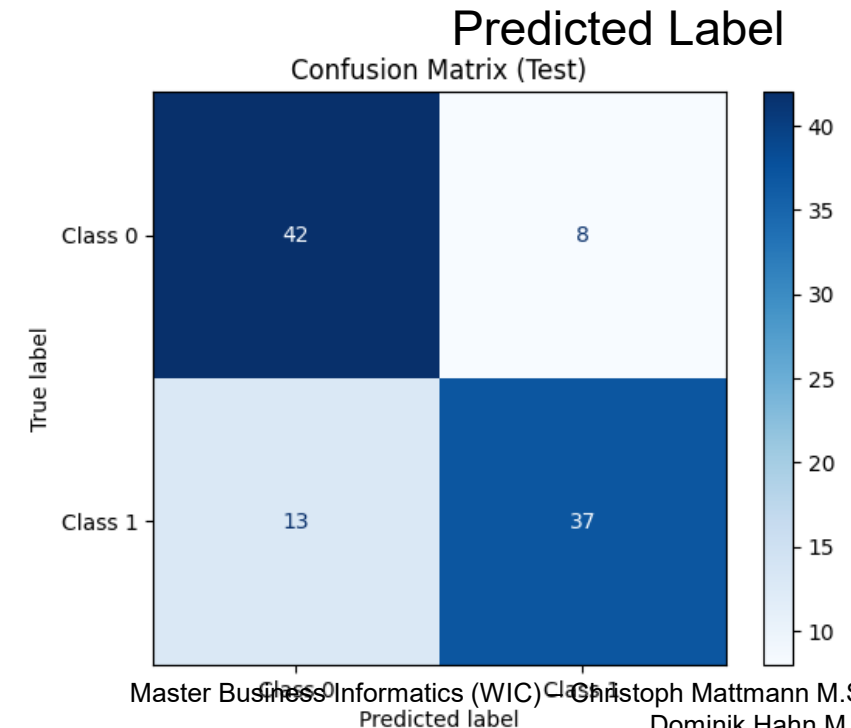
$$\text{Recall/Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$F1 = \frac{2 * \text{TP}}{2 * \text{TP} + \text{FP} + \text{FN}} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

True Label	Positive	True Positive (TP)	False Positive (FN)
	Negative	False Negative (FN)	True Negative (TN)
		Positive	Negative



Naïve Bayes Classifier

Naiv , ob es eine Verbindnug zwischen gratis und Gewinn besteht.

$$p(\text{spam/email}) = ((\text{gratis/Spam} * \text{Gewinn/Spam}) * \text{Spam}) / p(\text{mails})$$

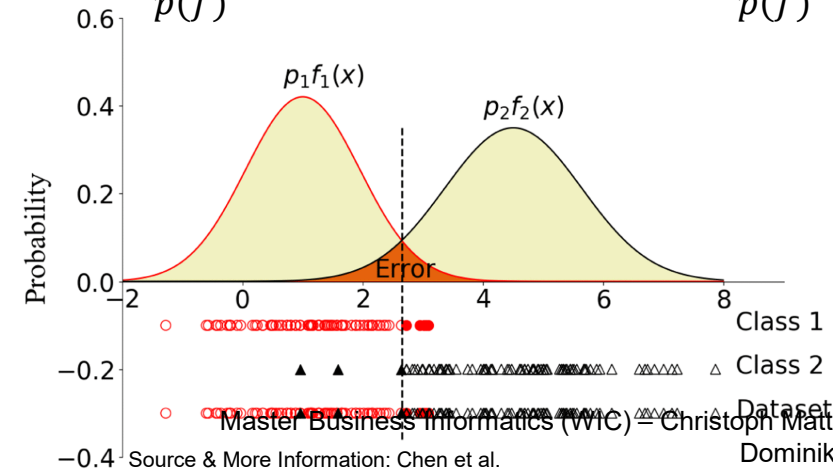
The **Bayes theorem** is used to determine the decision function:

$$p(\omega_1|f) = \frac{p(f|\omega_1) * p(\omega_1)}{p(f)}$$

$p(\omega_1 f)$	posterior probability, i.e. the probability that when observing a feature value of f , the instance is of class ω_1
$p(f \omega_1)$	probability density function (pdf), i.e. the probability of observing the feature value f in instances of the class ω_1 ,
$p(\omega_1)$	prior probability that an instance of class ω_1 will be observed in the data set
$p(f)$	prior probability that a given feature value be observed

For classification, the posterior probabilities $p(\omega_i|f)$ for an observed feature value are compared.

$$p(\omega_1|f) = \frac{p(f|\omega_1) * p(\omega_1)}{p(f)} \quad ? \quad > \quad p(\omega_2|f) = \frac{p(f|\omega_2) * p(\omega_2)}{p(f)}$$



Naïve Bayes Classifier

Wichtig Wissen über Wahrscheinlichkeitsverteilung

Advantages:

- Interpretability: well-known mathematical foundation.

Disadvantages:

- Robustness: assumes independence between the features (“class conditional independence”).
- Requires knowledge/estimation of the probability density function for each feature (type and parameters) and the prior probabilities.

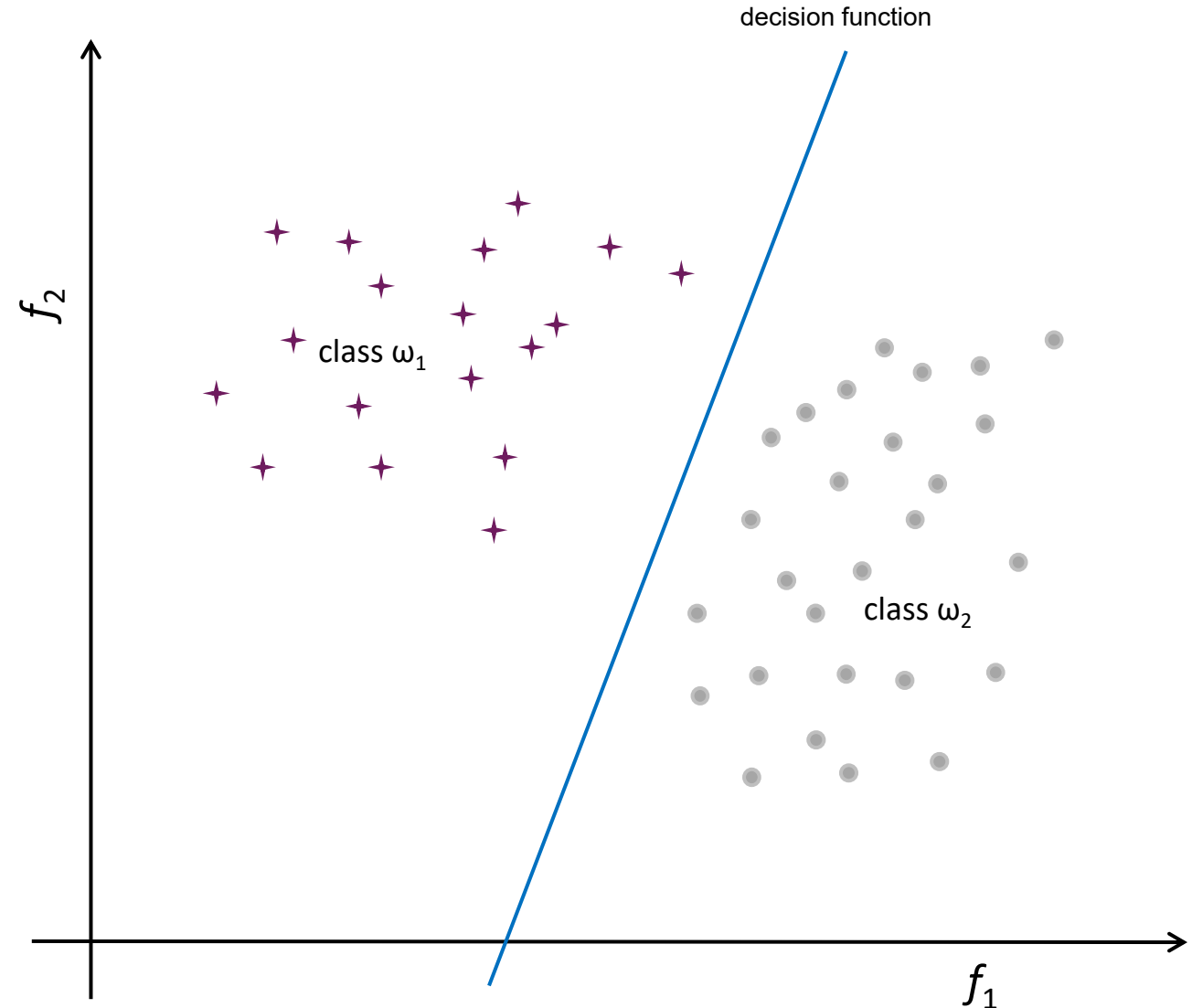
Linear Classifier

Zwei Klassen mit Decision Function zu trennen

General description of linear classifiers

- the idea:
separate classes with a linear decision function

So eindeutige Trennung mögl. der Klassen.

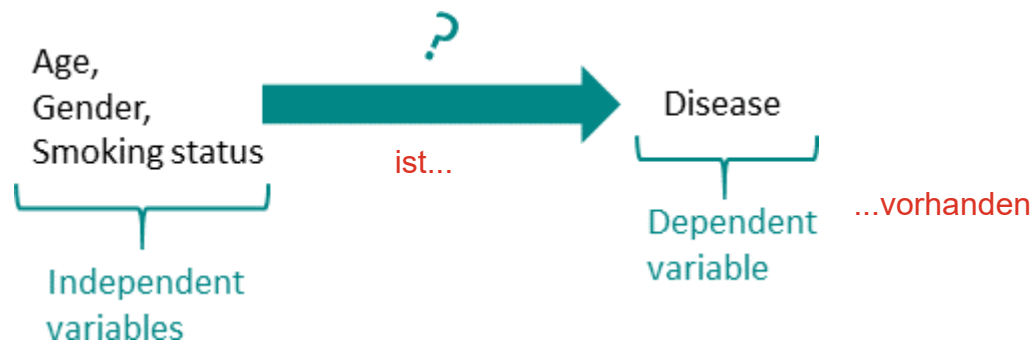


Logistic Regression

- **Binomial** Logistic Regression: This type is used when the dependent variable has only two possible categories. Examples include Yes/No, Pass/Fail or 0/1. It is the most common form of logistic regression and is used for binary classification problems. *Nutzung, meistens für Erklärungen*
Meist für binary classification
- **Multinomial** Logistic Regression: This is used when the dependent variable has three or more possible categories that are not ordered. For example, classifying animals into categories like "cat," "dog" or "sheep." It extends the binary logistic regression to handle multiple classes.
Mehr als 2 Klassen. Ohne Reihenfolge
- **Ordinal** Logistic Regression: This type applies when the dependent variable has three or more categories with a natural order or ranking. Examples include ratings like "low," "medium" and "high." It takes the order of the categories into account when modeling.
Klassifikation mit Hilfe eines Rankings

Logistic Regression

- Common method is Binomial
- It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1.
- It uses sigmoid function to convert inputs into a probability value between 0 and 1.
- For more than two categories SoftMax functions are used.



Understanding Sigmoid

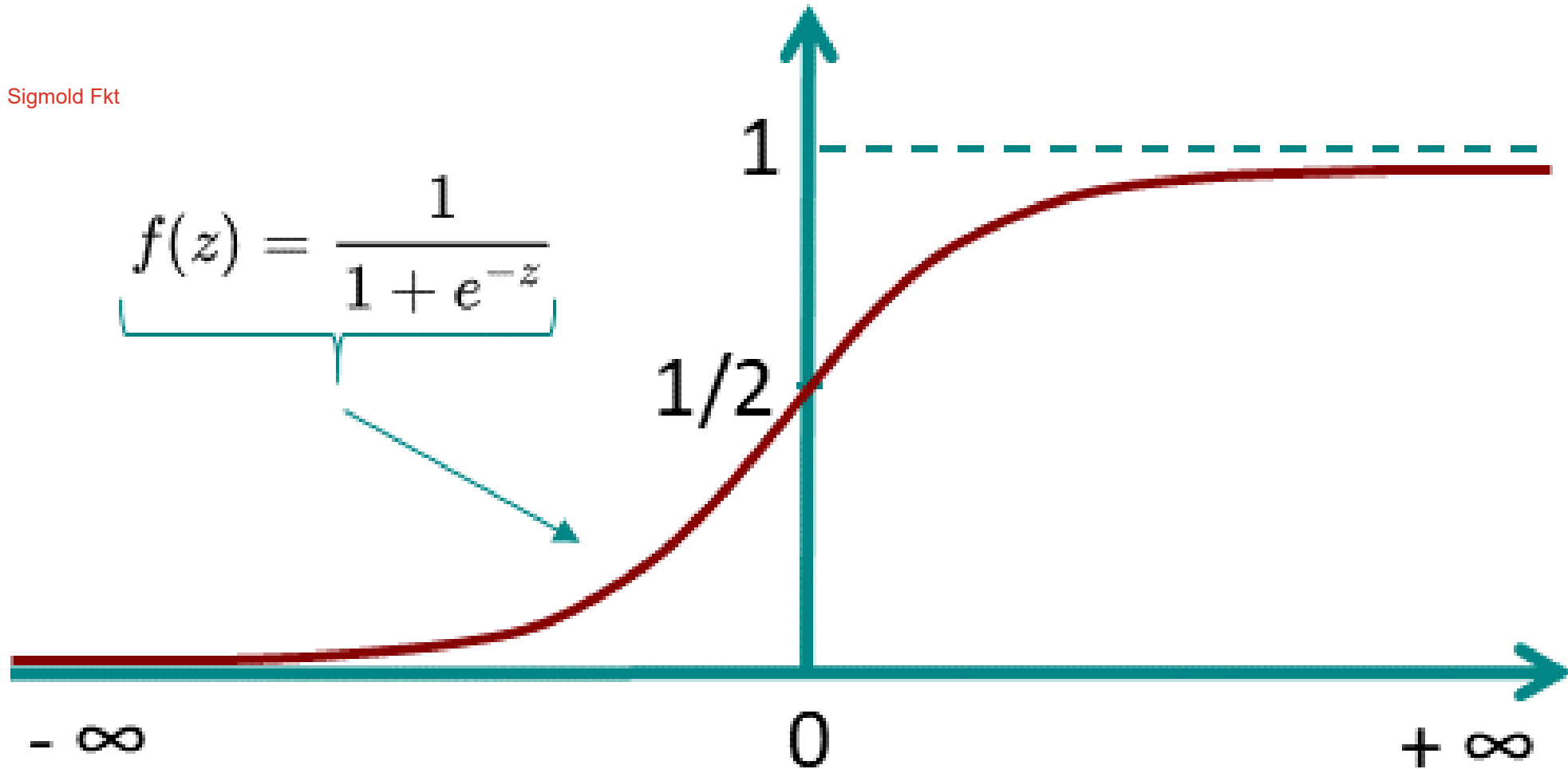
- This function takes any real number and maps it into the range 0 to 1 forming an "S" shaped curve called the sigmoid curve or logistic curve. Because probabilities must lie between 0 and 1, the sigmoid function is perfect for this purpose.

0,5 => Standardwert --> für Optimum für die Daten => Anpassung des threshold
--> durch Datta-Knowledge

- In logistic regression, we use a **threshold value** usually 0.5 to decide the class label.
 - If the sigmoid output is same or above the threshold, the input is classified as Class 1.
 - If it is below the threshold, the input is classified as Class 0.

Logistic Regression

Sigmoid Fkt



Logistic Regression

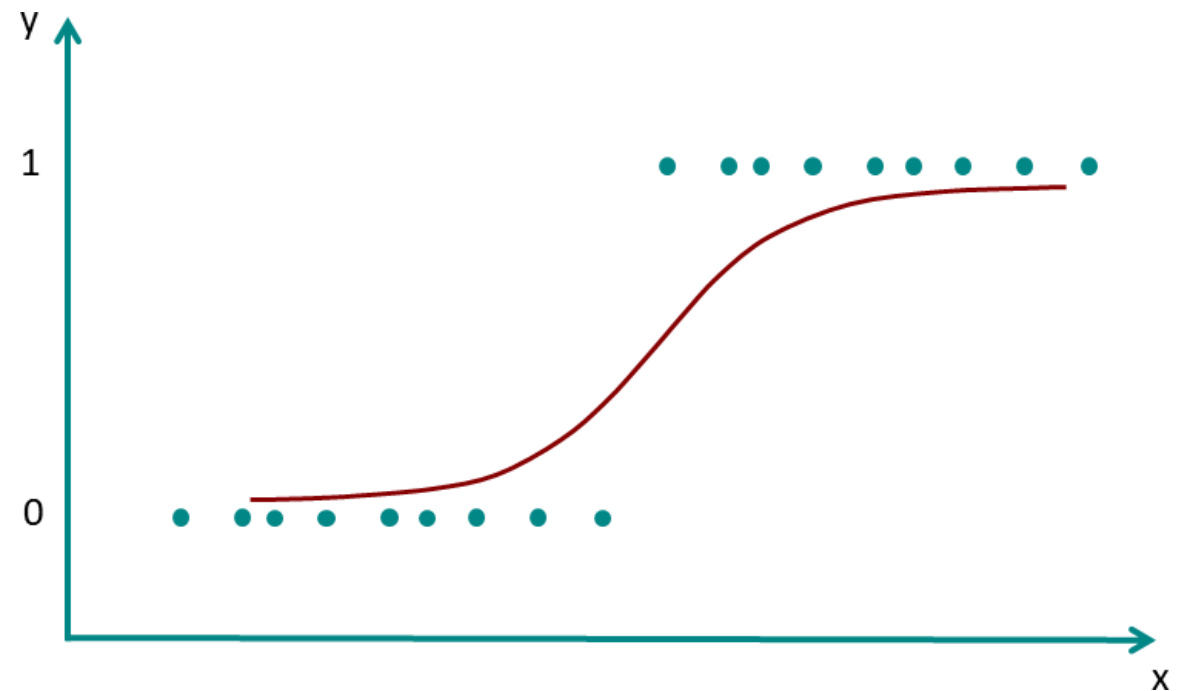
- To calculate for example disease probability, the model parameters b_1 , b_2 and b_3 , a must first be determined. Once these have been determined, the equation for the example is:

Regularizations => sind die b-Values

$$P(y = 1 \mid x_1, \dots, x_n) = \frac{1}{1 + e^{-(b_1*x_1 + \dots + b_n*x_n + a)}}$$

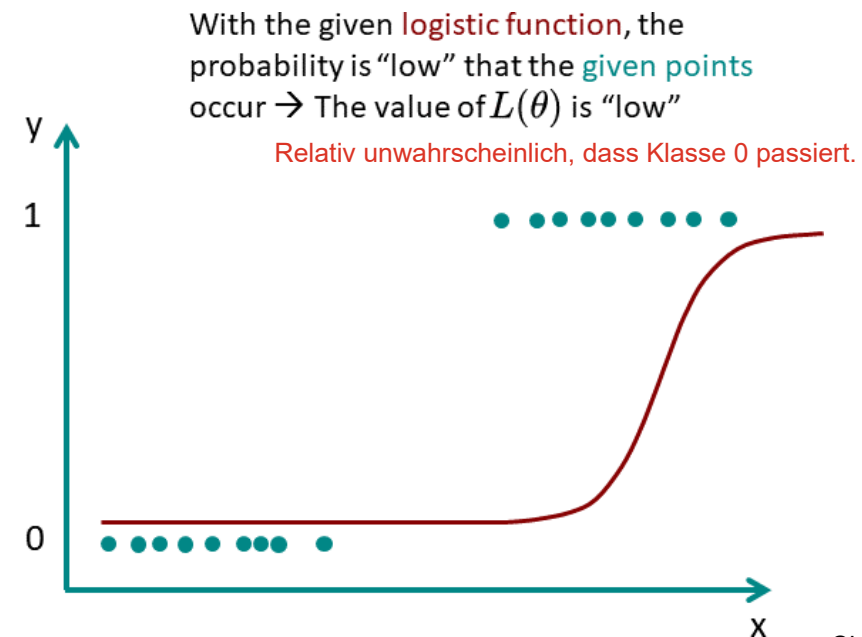
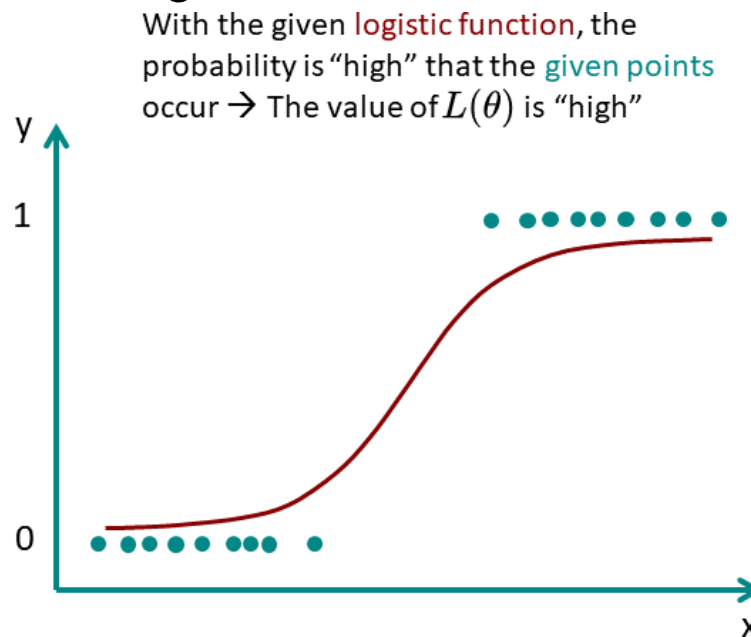
$$P(y = 0 \mid x_1, \dots, x_n) = 1 - \frac{1}{1 + e^{-(b_1*x_1 + \dots + b_n*x_n + a)}}$$

$$P(\text{Diseased}) = \frac{1}{1 + e^{-(b_1*Age + b_2*Gender + b_3*Smoking + a)}}$$



Logistic Regression

- To determine the model parameters for the logistic regression equation, the **Maximum Likelihood Method (L)** is applied. L is a function of the unknown parameters in the model, in case of logistic regression these are $b_1, \dots, b_n, a$. Therefore we can also write $L(b_1, \dots, b_n, a)$ or $L(\theta)$ if the parameters are summarized in θ .
- $L(\theta)$ indicates how likely it is that the observed data occur. As θ changes, the probability of the observed data changes.



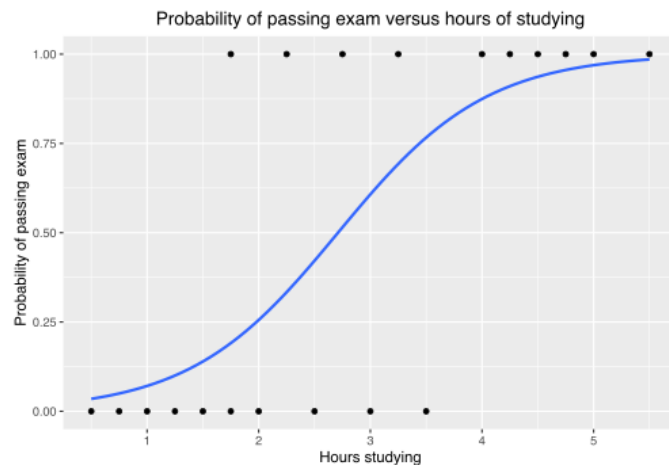
Logistic Regression

- Advantages:
 - easy to interpret
 - fast to train
 - outputs probabilities
 - strong baseline for many classification tasks gute Erg. ohne viel Aufwand
- Disadvantages:
 - only linear decision boundaries
 - limited performance on complex non-linear data
 - sensitive to strongly correlated features / outliers
 - often requires feature scaling in practice

Logistic Regression

Regularizations => sind die b-Values

- some important parameters:
 - penalty: regularization type (default: L2 regularization) Type machen damit Parameter nicht zu groß und alle in Betracht gezogen werden
 - C: inverse regularization strength; smaller values mean stronger regularization (default: 1.0) Wie stark soll regularisiert werden
 - solver: optimization algorithm (default: lbfgs)



ZB so in Prüfung:

```
# split into training and test data Könnt ihr ein Train Test Split anlegen
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=123
)
```

```
# scale training and test data
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

```
# create and train logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
# predict labels for test data
y_pred = model.predict(X_test)
```

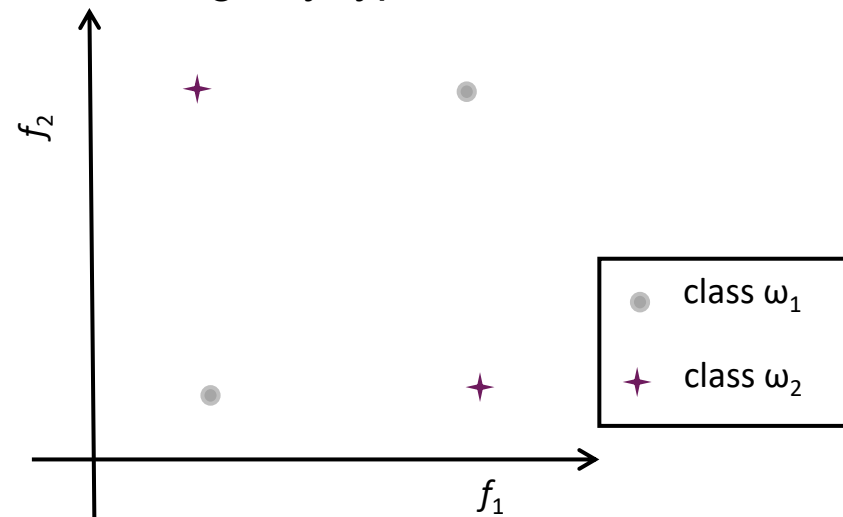
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Non-linear Classifier

Dass Daten nicht linear getrennt werden können.

„XOR-Problem“

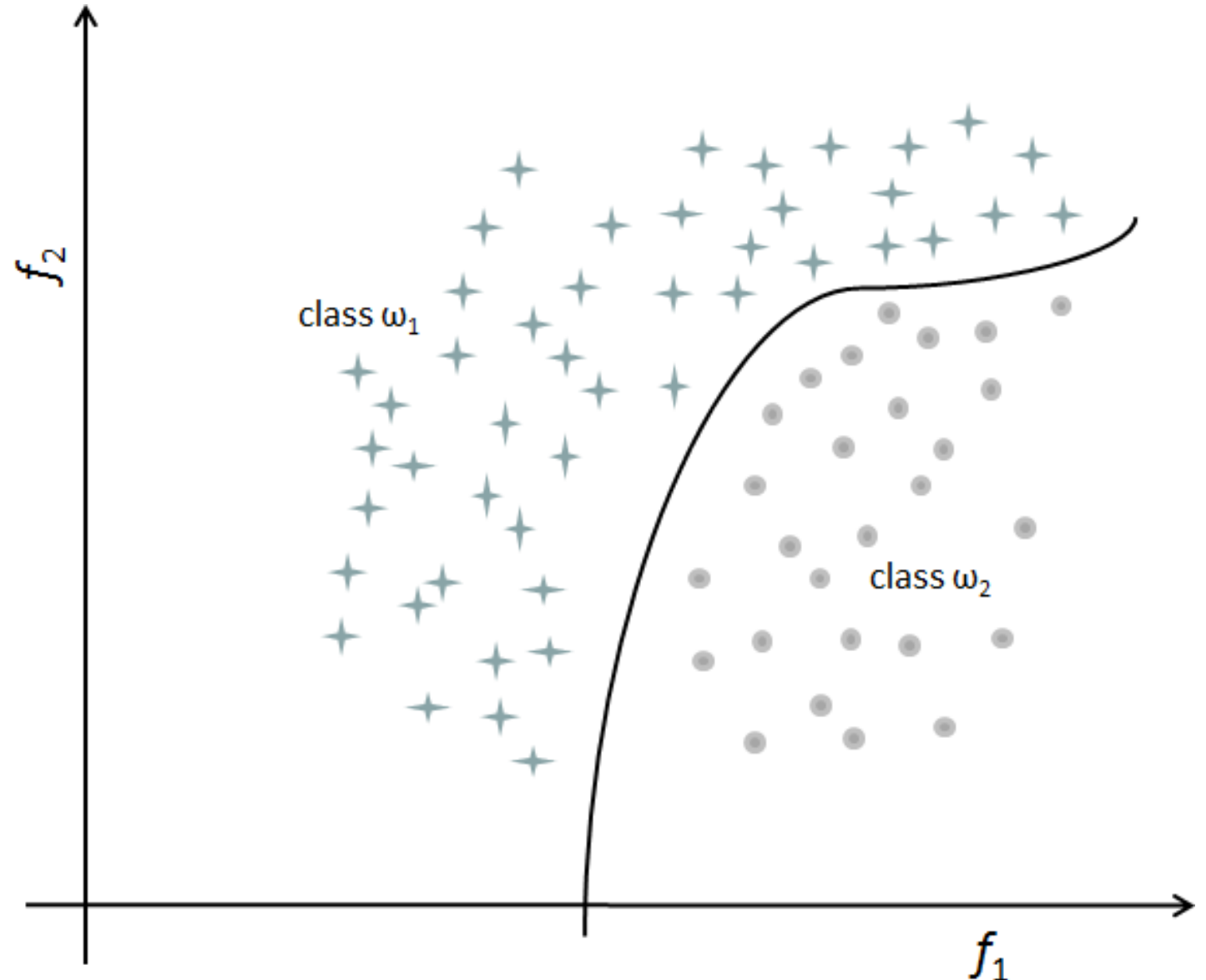
- Linear classifiers have a strong limitation:
 - They work well when the classes are linearly separable, if this is not the case they can't find a good solution
- Let's have a look at the so-called “XOR-problem”:
 - A data set with only four instances that appears to be quite simple
 - Try to separate the two classes using any type of linear classifier



Non-linear Classifier

Introducing non-linear classifiers

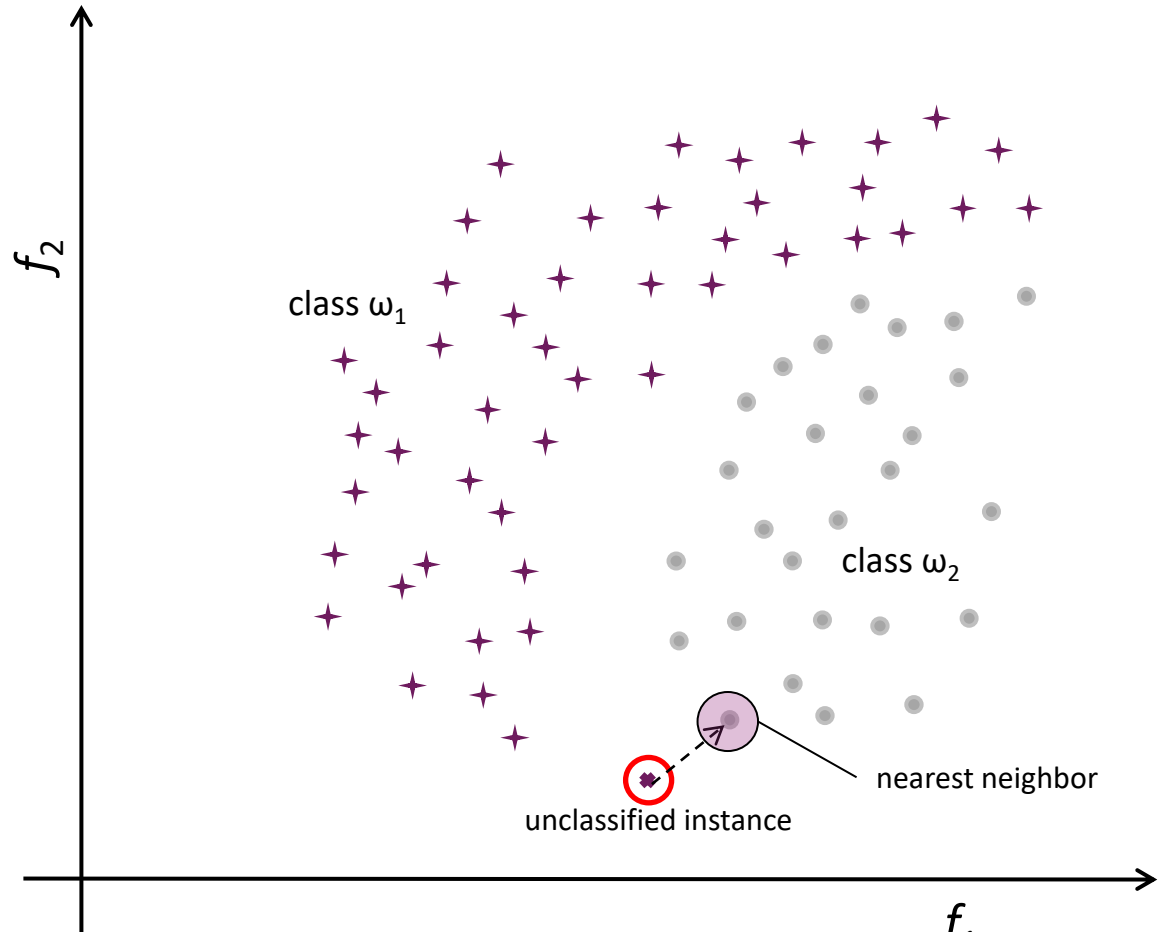
- to overcome the problem of linearly non-separable data sets:
 - classifiers with non-linear decision boundaries are introduced in the following slides



Nearest Neighbor (NN, 1-NN)

- The idea:
 - an instance belongs to the class of its nearest neighbor
- How it works:
 - store all instances from a training set
 - for an unclassified instance find the nearest neighbor in the training set
 - assign the class of the nearest neighbor
- Advantages
 - Easy interpretable and to implement
 - Can be used for any data
 - Extremely fast (just data storing)
- Disadvantage
 - Sensitive to outliers
 - Scalability (revisits every instance)

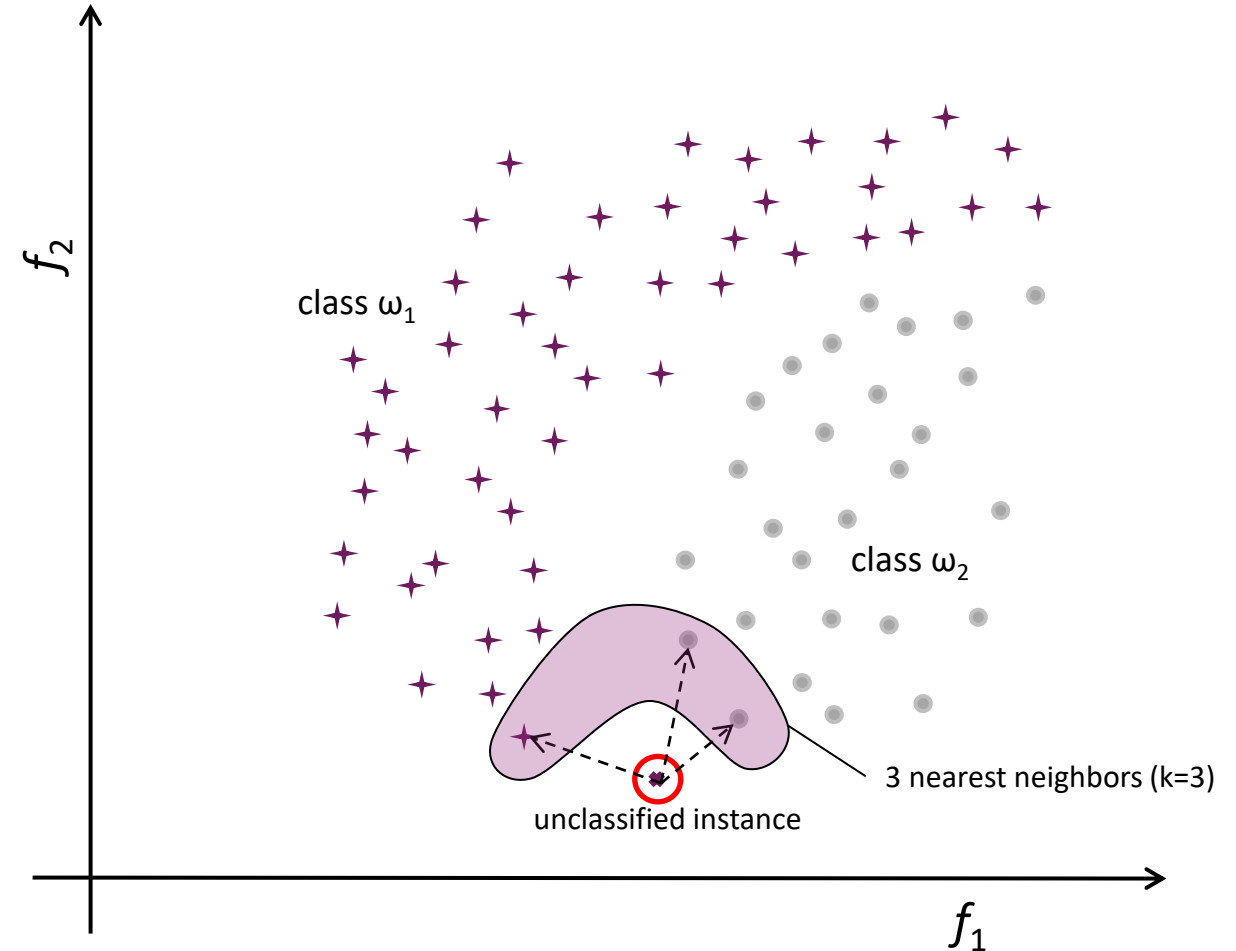
Welche Klasse hat der nächstmögliche Pkt., deshalb muss dieser auch diese Klasse haben.



k-Nearest Neighbors (k-NN)

Man schaut nicht auf den einen nächsten Nachbarn, sondern auf z.B. die 5 nächsten.

- The idea:
 - an instance belongs to the class of its k nearest neighbors
- How it works:
 - store all instances from a training set
 - select the parameter k
 - for an unclassified instance find the k nearest neighbor in the training set
 - classify the instance based on the majority class in the k nearest neighbors (typically odd numbers are used: 3, 5, 7, ...)
- Removes the disadvantage of being sensitive to outlier



Die Modelle grob als Code kennen können

k-Nearest Neighbors (k-NN)

```
# create dataset
```

```
X, y = make_moons(n_samples=500, noise=0.1, random_state=123)
```

```
# split into training and test data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5)
```

```
# fit scaler on training data only
```

```
scaler = MinMaxScaler().fit(X_train)
```

```
# scale training and test data
```

```
X_train = scaler.transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Er macht auch Lückenfüllaufgaben

```
# create and train k-NN model with k=3 (Bsp. in der Prüfung)
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

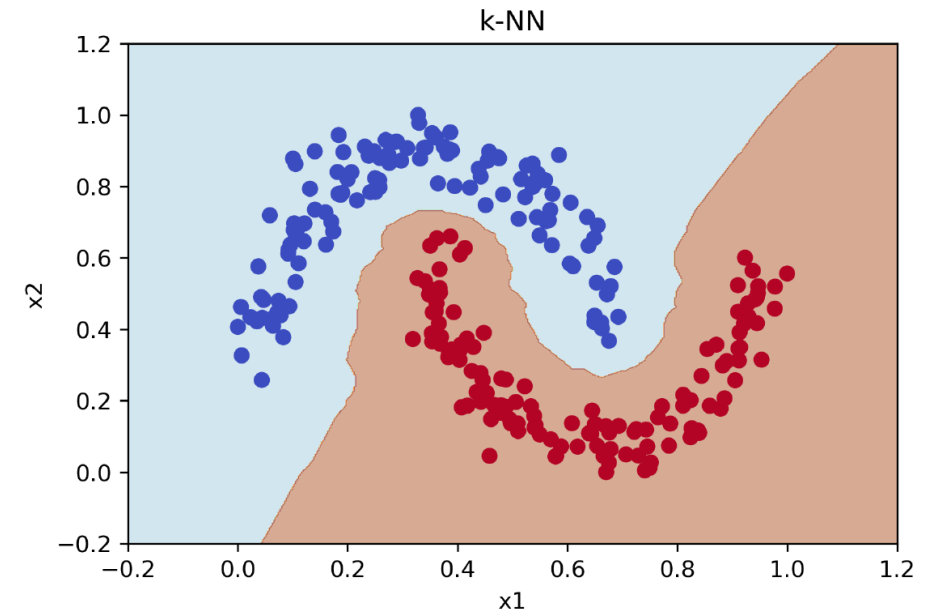
```
model.fit(X_train, y_train)
```

```
# predict labels for test data
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print(confusion_matrix(y_test, y_pred))
```



Decision Tree

- The idea:

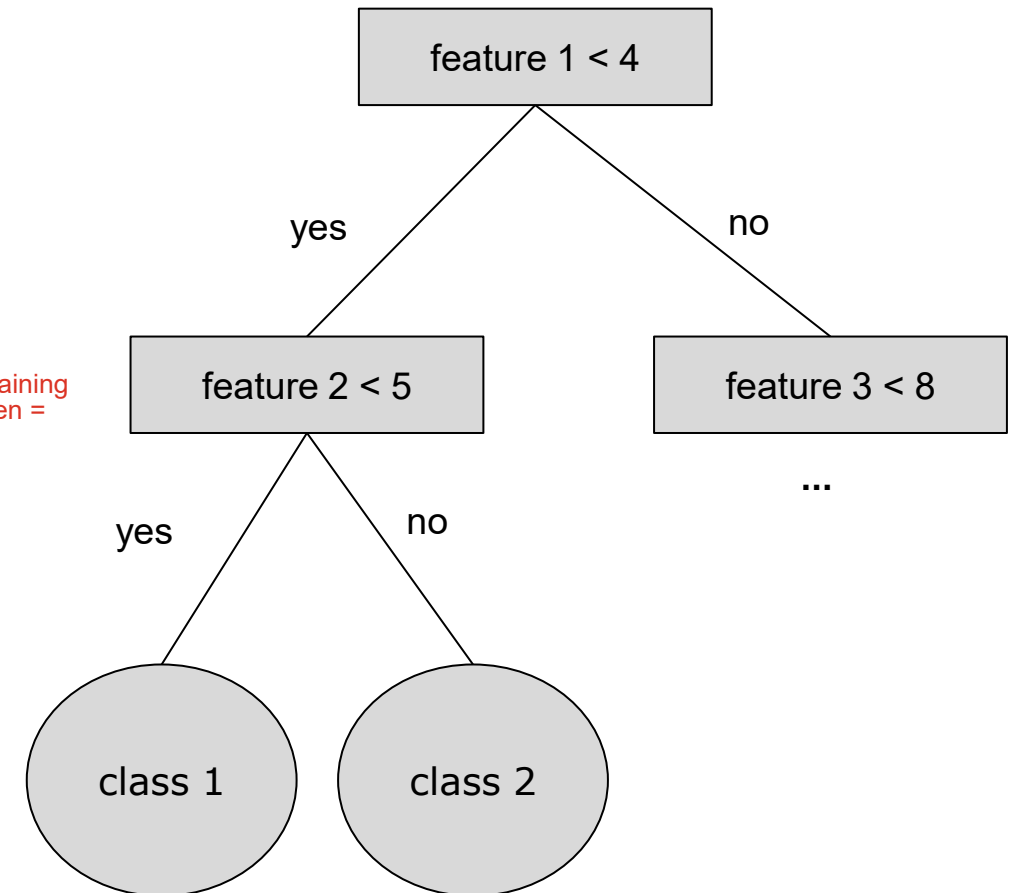
- Separate the classes in the training set by recursively splitting the data by thresholding individual features.

=> Versuch das data set anhand von features regelbasiert zu unterteilen.

- The algorithm:

- Create a node
- Select the best split using some splitting criterion
- Grow branches according to the split (binary vs. multiple branches)
- Recursively continue with 1. until some stopping criterion is met or all branches contain only feature vectors from one class („purity“) → bottom nodes become „leaves“

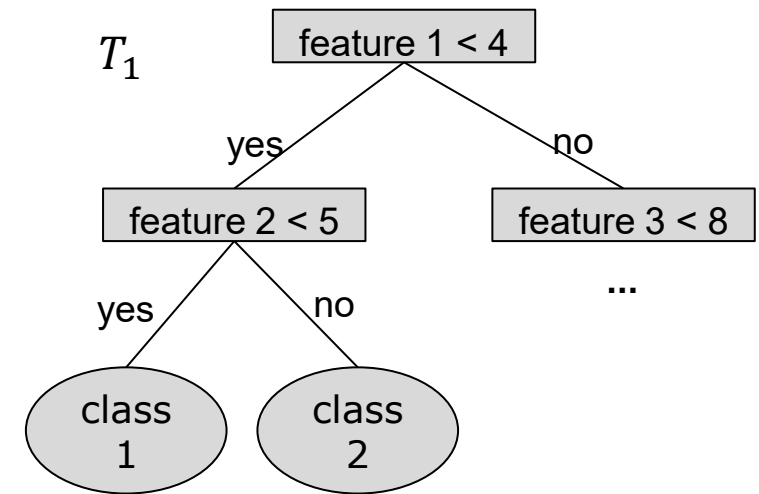
Splitting-Kriterien im Training
(Kriterien hier am Knoten =
ein Trainingswert)



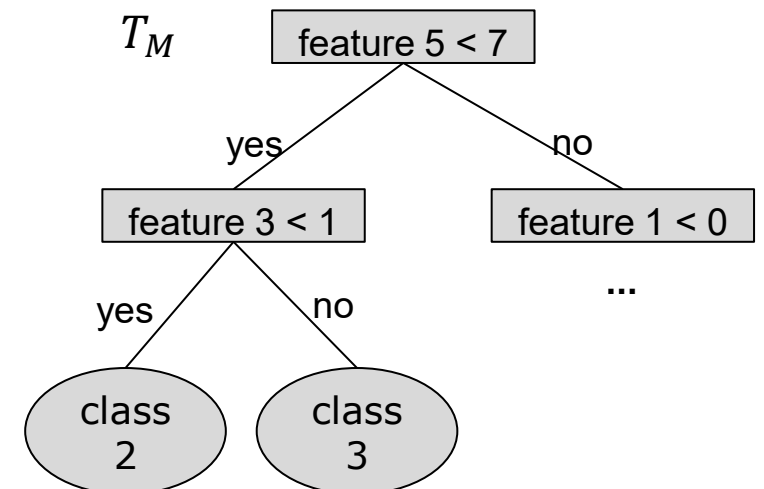
Random Forest

Eine Ensemble Methode

- The idea:
 - Create many decision trees and combine the result with majority voting.
- Assumption: the result of a combination of different classifiers is likely to be better than of a single classifier.
- A random forest is an ensemble method, i.e. a random forest is an ensemble of trees.
 - the number of trees is a hyperparameter (e.g. 100)
- The algorithm:
 - create many different decision trees $T_1 \dots T_M$
 - the different trees are created by randomly subsampling the features and the data set at each node of each tree
 - a feature vector in the test set is classified by each tree T_i
 - the classification result is the most frequent result from $T_1 \dots T_M$ (majority voting)



...



Random Forest

- An ensemble performs best, **if the base classifiers are diverse and unrelated**. --> bagging =>
- In a random forest this is achieved by:
- **bagging**: training each of the decision trees with a different subset of the data
 - bootstrap aggregation (bagging) is typically used for this
 - in bagging, from a data set D consisting of N feature vectors, N of these feature vectors are randomly drawn with replacement and become the training set A
 - i.e. a feature vector can be drawn multiple times
 - the feature vectors not drawn become the test set B
- **feature subsampling**: randomly selecting subsets of features at each node of each tree
 - for example for $|F|$ features, $\sqrt{(|F|)}$ can be selected as candidates at each node
 - using a splitting criterion, the best split is determined from this subset

nicht für jeden Tree alle Attribute vorhanden (jeder Des. Tree wird je spezialisiert)

Random Forest

- Advantage

- avoids overfitting Weil nicht immer jedes Feature mit gegeben
- reduces the variance
- often better results compared to single trees
- works with numerical and categorical data
- no scaling of input data required
- works for large, high-dimensional data sets, since at each split only a subset is tested
- can return feature importance Welches Feature war nach Klassifizierung am wichtigsten --> ermöglicht dass Random Forest ein Baseline Modell ist.
- can also be used for regression

- Disadvantage

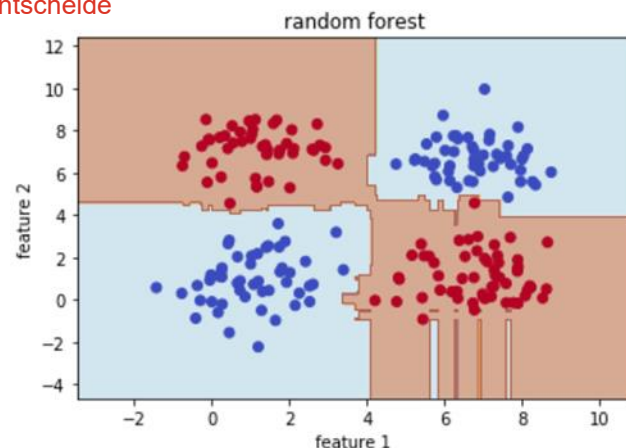
- in contrast to single decision trees, a random forest is not (easily) interpretable
- computationally expensive, compared to single tree

Random Forest

- some important parameters:
 - `n_estimators`, determining the number of decision trees to be created
 - `max_depth`, allowing to prune the tree
 - `max_features`, number of features to consider at each split
- current defaults are: `n_estimators=100`, `criterion='gini'`, `max_depth=None`

Merkmale an denen ich entscheide

Wie lange darf es dauern bis ich zu Entscheidung schaffe



```
# create dataset
```

```
data, labels = createData_XOR()
```

```
# split data set into train and test set
```

```
train_data, test_data, train_labels, test_labels =  
    train_test_split(data,  
                    labels,  
                    test_size=0.5,  
                    random_state=123)
```

```
# training
```

```
model = RandomForestClassifier(random_state=123)  
model.fit(train_data, train_labels)  
print(model)
```

```
# classification of test set
```

```
predictions = model.predict(test_data)  
cm = confusion_matrix(test_labels, predictions)  
print(cm)
```

Gradient Boosting

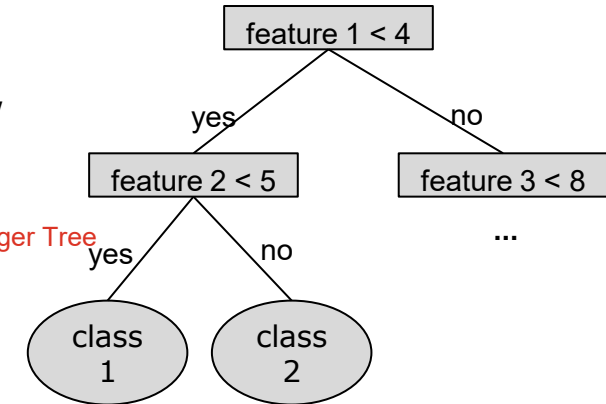
schaut ein Baum an und den Error den er predited und den nächsten Baum darauf optimiert.

- Gradient boosting machines (GBM) refers to a family of methods of ensembles of trees using boosting
- GBMs are also referred to as gradient tree boosting
- An early boosting algorithm with decision trees is AdaBoost (Adaptive Boosting)
- Later generalized and then termed Gradient Boosting Machines (Friedmann, 2000)
- A widely used scalable implementation is XGBoost (Extreme Gradient Boosting) (Chen and Guestrin, 2016)

XGBoost (Extreme Gradient Boosting)

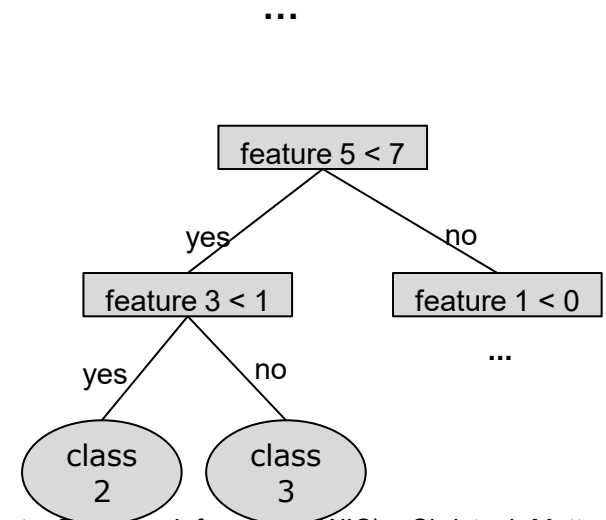
- The idea:

- Create many **sequential decision trees** $T_1 \dots T_M$, where each new tree T_i tries to minimize the errors of the previous tree $T_{(i-1)}$.
Minimierung jeden nächsten Trees, abhängig des vorherigen Trees
- Combine the results with weighted voting.
Letzte Tree höhere Des. Tree als der vorheriger Tree
- XGBoost is an ensemble method consisting of sequential trees.



- The algorithm:

- add one tree T_i per step
- the new tree is found such that the error of the previous tree $T_{(i-1)}$ is minimized
- Got to 1. until some stopping criterion is reached (e.g. number of steps)
- the classification result is found by using the results of $T_1 \dots T_M$, weighted by their accuracies



XGBoost

- Advantages

- avoids overfitting
- usually better results compared to single trees
- works with numerical and categorical data
- can handle missing values (lieber immer selber davor machen)
- no scaling of input data required
- works for large, high-dimensional data sets, since at each split only a subset is tested
- can return feature importance

- Disadvantages

- in contrast to single decision trees, a XGBoost is not (easily) interpretable
- computationally expensive compared to single trees, however XGBoost was implemented towards efficiency and scalability

XGBoost

nutzung subsets

- some important parameters:
 - `n_estimators`, determining the number of decision trees to be created
 - `max_depth`, allowing to prune the tree
 - `reg_lambda`, regularisation parameter controlling the trade-off between loss and reg. term
 - `learning_rate`, also referred to as η . controls the contribution of each new tree (0,1)
- Current defaults are: `n_estimators=100`, `max_depth=6`, `reg_lambda=1`, `learning_rate=0.3`

```
# create dataset
```

```
data, labels = createData_XOR()
```

```
# split data into train and test set
```

```
train_data, test_data, train_labels, test_labels =  
    train_test_split(  
        data, labels, test_size=0.5, random_state=123  
    )
```

```
# training
```

```
model = XGBClassifier()  
model.fit(train_data, train_labels)  
print(model)
```

```
# classification of test set
```

```
predictions = model.predict(test_data)  
cm = confusion_matrix(test_labels, predictions)  
print(cm)
```

Support Vector Machines (SVM)

x_i : one dimension in the feature space, i.e. one feature

$\vec{x}_i$: one feature vector (one instance)

$\vec{x}$: the feature space

$\vec{x}_i \vec{x}_j$: inner product, dot product, scalar product of $\vec{x}_i$ and $\vec{x}_j$

N : number of instances

n : number of features

$\vec{w}$: vector of weights

$||\vec{w}||$: norm of the vector $\vec{w}$

ω_i : class i

y_i : label (+1 or -1)

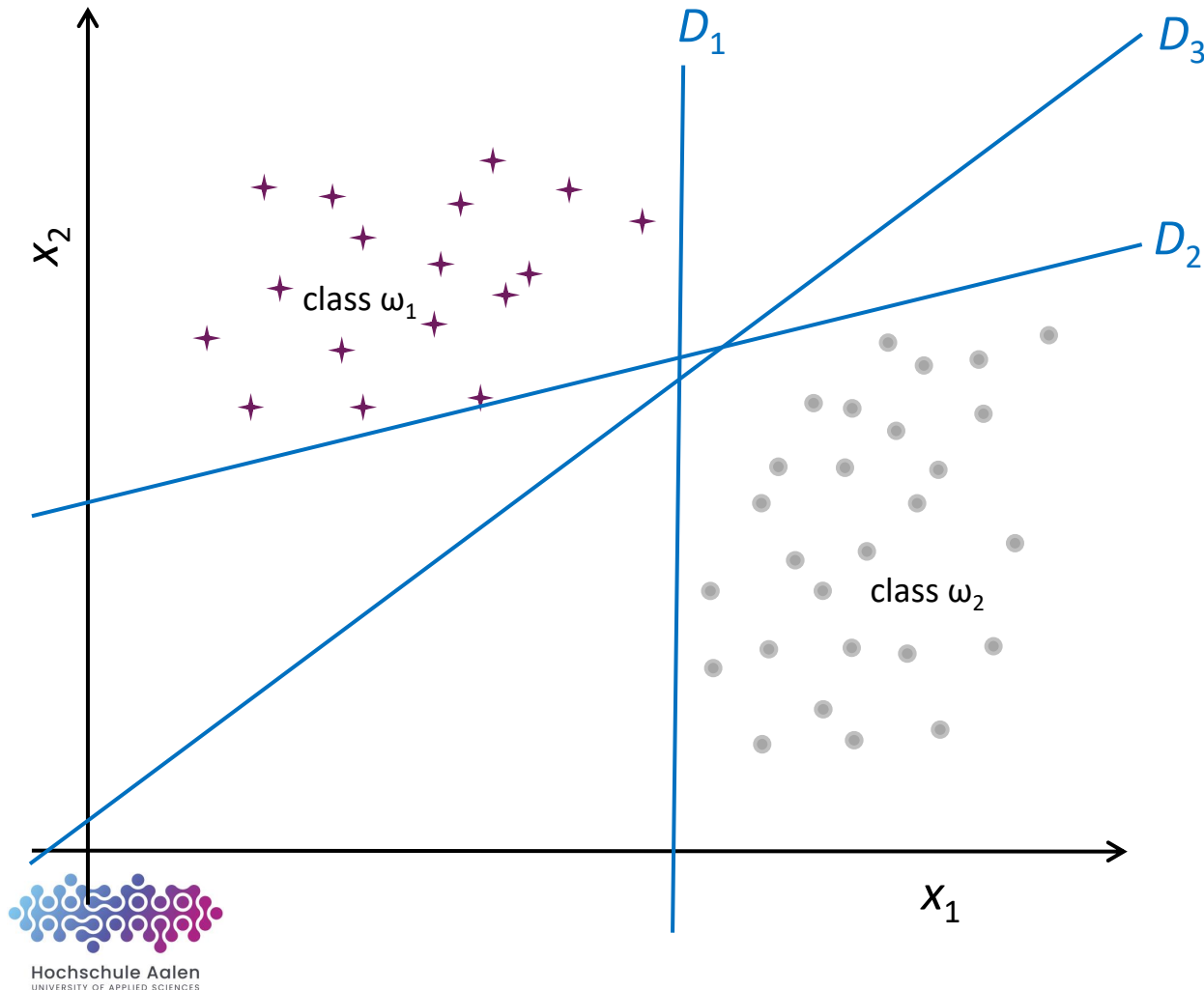
We use arrows to denote vectors, in contrast to most of the literature on data mining, where vectors are written without arrows.

Support Vector Machine (SVM)

Klassen trennen

D_3 gewählt --> weil hier passender Abstand zu den Mengen

Separating two classes with linear decision functions



The number of possible decision functions is infinite.

So which one should be chosen?

- Intuitively D_3 seems to be the best, but why?

Support Vector Machines (SVM)

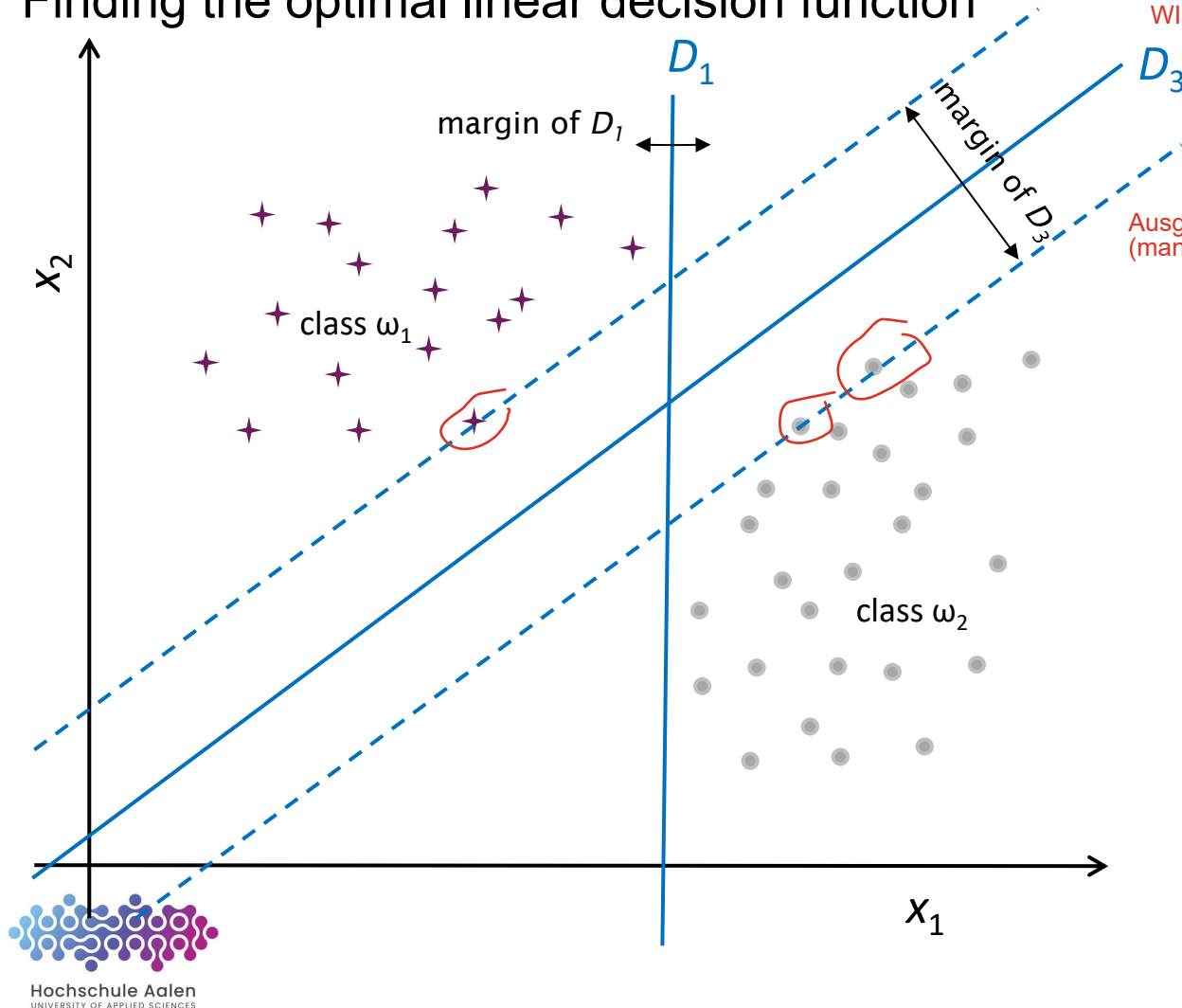
Erklärt wie SVM funktioniert Siehe auch Folie 39

- The idea:
 - separate the instances of two classes using a linear decision function referred to as “hyperplane”
- How it works: (informal explanation for a two-dimensional space)
 - find two parallel lines, one intersecting one or more instances at the boundary of class ω_1 and the other line intersecting one or more instances of class ω_2
 - find the line, with equal distance to each of the two parallel lines (the line „in the middle“)
 - measure the distance between the two outer lines
 - this distance is referred to as the „margin“
 - the optimal decision function is the one with the maximum margin
 - the decision function is expressed using instances from the training set, the so-called support vectors

Support Vector Machines (SVM)

Versucht mit Support Vektoren die Des. Function zu finden

Finding the optimal linear decision function



Wie wird D_1 , D_2 und D_3 festgelegt?

Based on the informal explanation, let us compare the two candidates D_1 and D_3

Ausgehend von D_3 wird parallel Geraden gezogen die durch äußersten Punkte der Mengen geht (man versucht so viele Punkte wie möglich mit zu nehmen)

Our intuition was correct:

- D_3 has the maximal margin, it is much larger than the margin of D_1
- D_3 is the optimal decision function

Support Vector Machines (SVM) nicht relevant

- A linear function in a two-dimensional space can be expressed by the well-known „slope-intercept-form“:

$$y=mx+b$$

- The $y(x)$ -form is constrained to two dimensions. To make the formula more generic we use the dimensions x_1 and x_2 and w_1 instead of m :

$$x_2 = w_1 x_1 + b$$

- which can be reformulated as

$$w_1 x_1 + w_2 x_2 + b = 0$$

- Substituting the scalars with the vectors $\vec{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}$ and $\vec{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$
- leads to the general form of the linear decision function:

$$\vec{w} \cdot \vec{x} + b = 0$$

Support Vector Machines (SVM) nicht relevant

- Up to now we have looked at a two-dimensional space (2 features), where the linear decision function is simply a line.
- In a three-dimensional space (3 features) the line becomes a plane and in higher-dimensional spaces (>3) it is referred to as a „hyperplane“.
- We will use the term hyperplane, independent of the number of dimensions.
- **The hyperplane is expressed as**

$$\vec{w} \vec{x} + b = 0$$

or

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = 0$$

with

$$\vec{w} = \begin{pmatrix} w_1 \\ \dots \\ w_n \end{pmatrix} \quad \text{and} \quad \vec{x} = \begin{pmatrix} x_1 \\ \dots \\ x_n \end{pmatrix}$$

where n is the number of features. (For all data points $\vec{x}$ becomes a matrix)

Support Vector Machine (SVM)

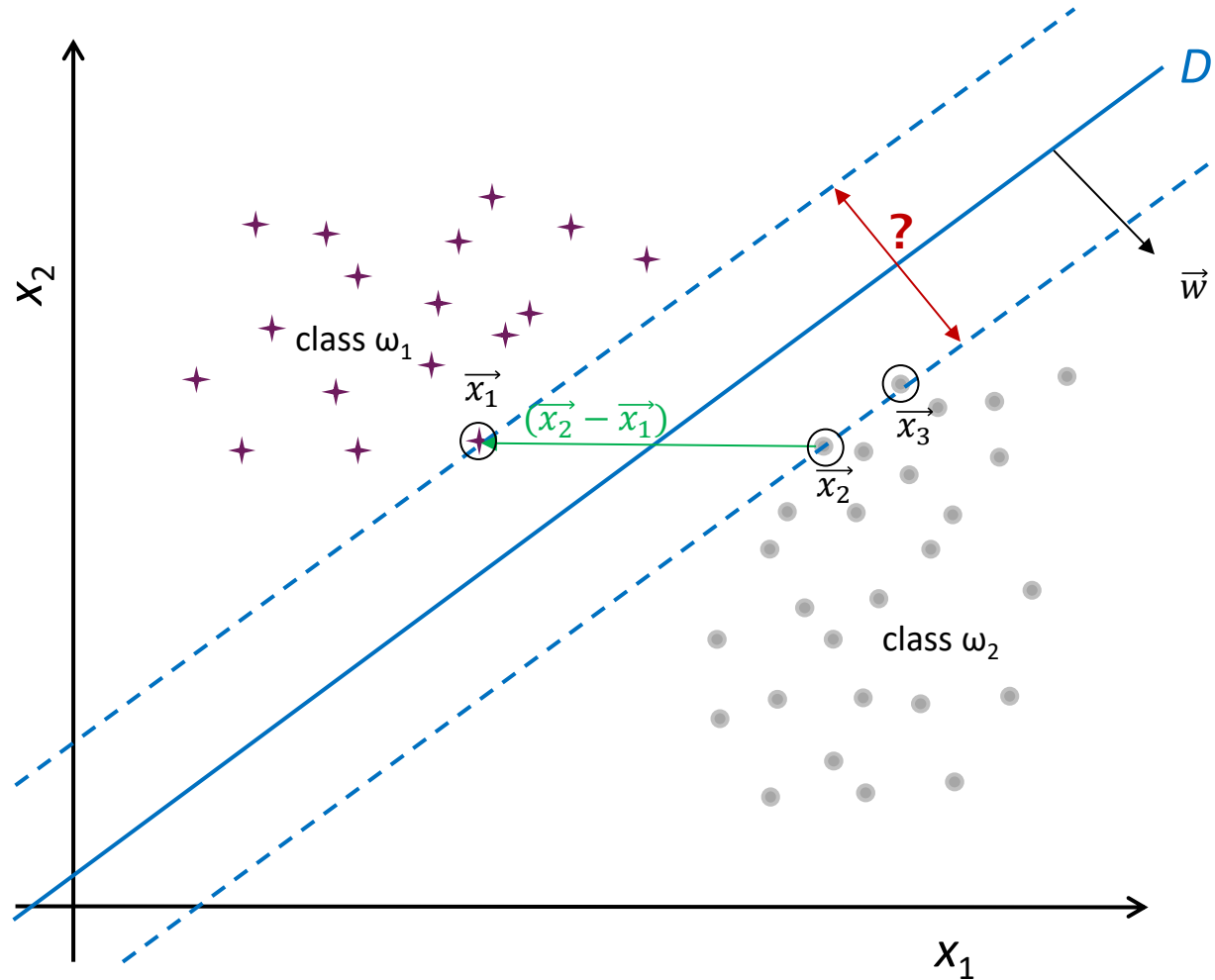
- The optimal **decision function** is **referred to** as the **maximum margin hyperplane** (MMH), as described with parallel lines for the two-dimensional case on the previous slides.
- In order to find the optimal hyperplane, we need to solve an optimization problem:

We want to maximize the margin given by $\frac{2}{||\vec{w}||}$ which is equivalent to minimizing $\frac{||\vec{w}||}{2}$

- A little mathematical trick: We minimize $\frac{1}{2} ||\vec{w}||^2$ instead of $\frac{1}{2} ||\vec{w}||$
- (Minimizing $\frac{1}{2} ||\vec{w}||$ yields the same solution as minimizing $\frac{1}{2} ||\vec{w}||^2$, but we don't have to calculate the square root in $||\vec{w}||$ and the optimization problem will benefit from the squared $||\vec{w}||$)

Support Vector Machine (SVM)

Erklärt wie SVM funktioniert Siehe auch Folie 39

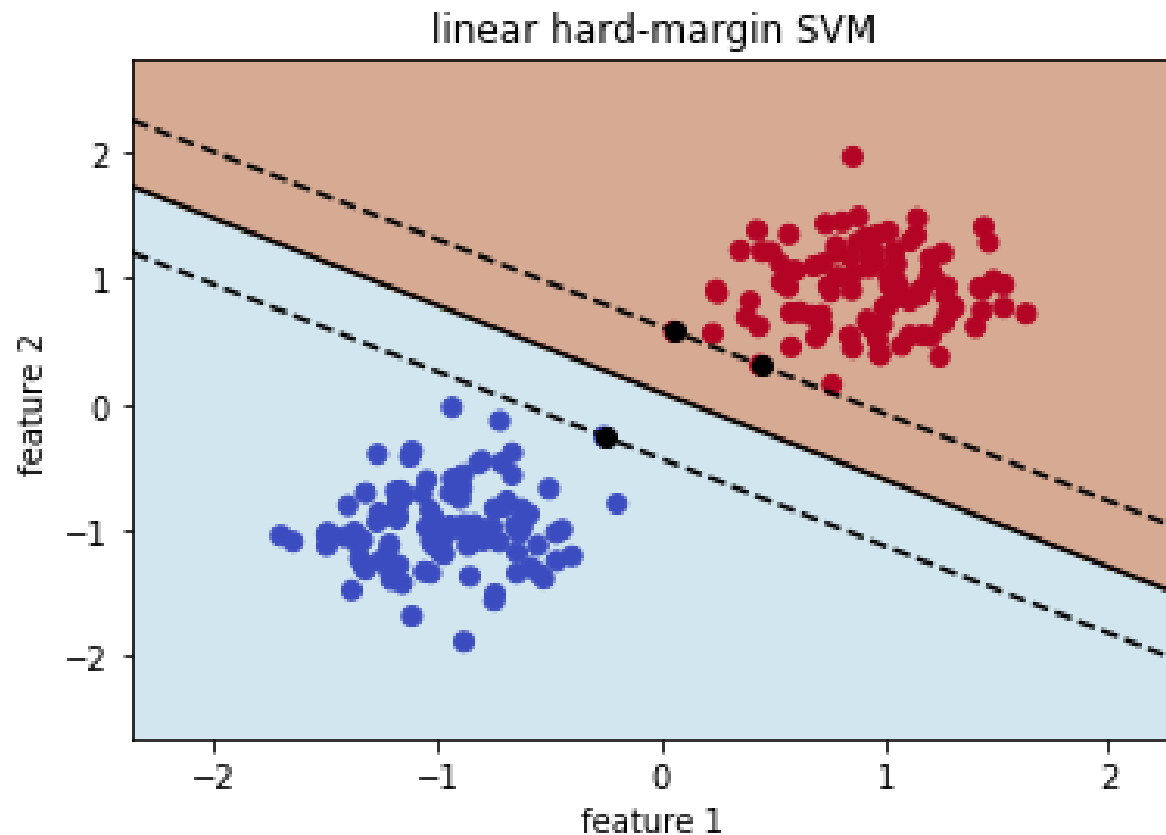


Since we want to maximize the margin, we first need to determine the width of the margin.

(the ? in the plot)

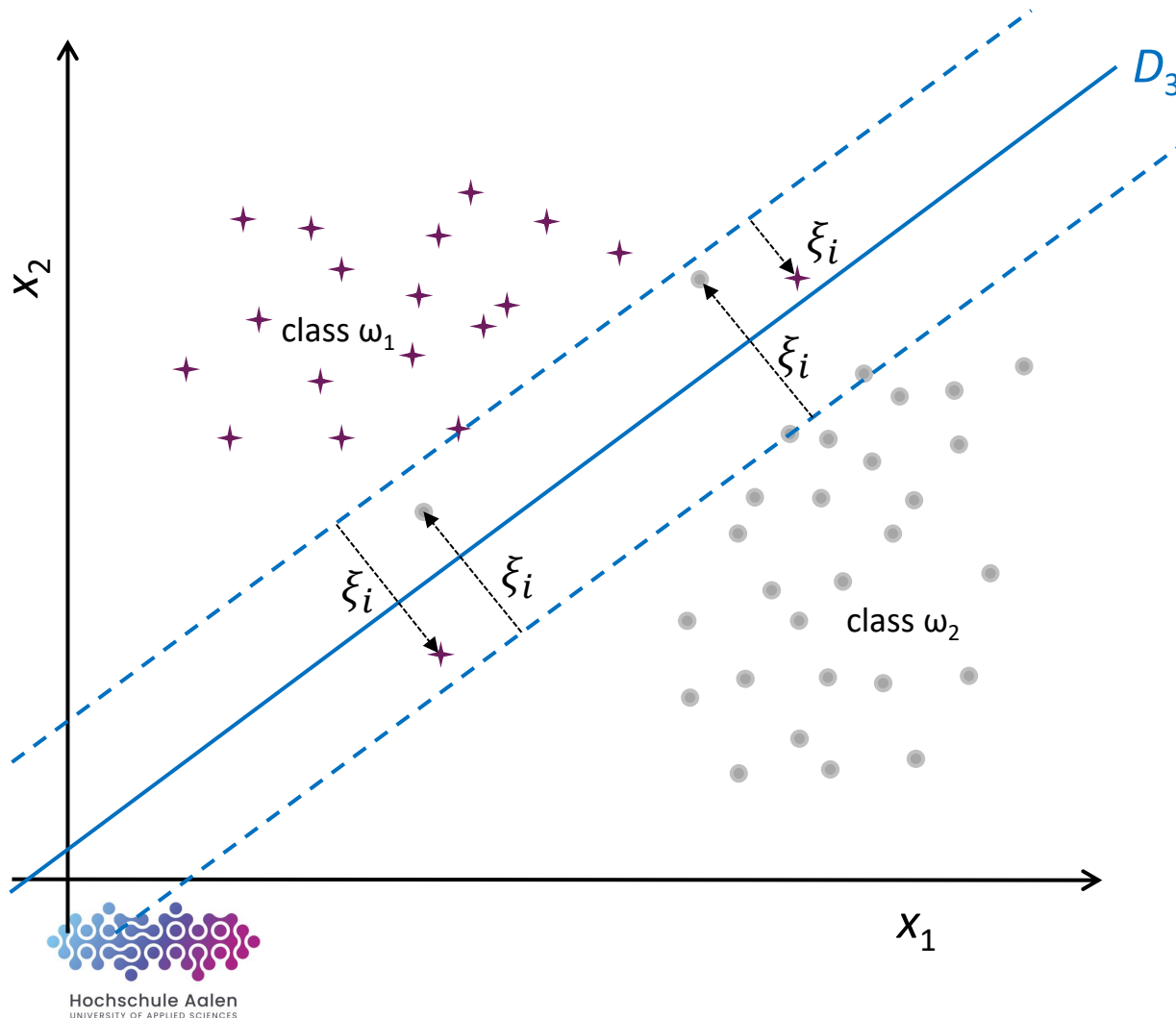
$\vec{x}_1, \vec{x}_2, \vec{x}_3$: support vectors
 $(\vec{x}_2 - \vec{x}_1)$: vector connecting $\vec{x}_2$ and $\vec{x}_1$
 $\vec{w}$: normal vector

Support Vector Machine (SVM)



- SVM finds the optimal linear decision function:
the „maximum margin hyperplane“
- The found hyperplane is guaranteed to be the optimal solution, there are no local minima
- The decision function is expressed using instances $(\vec{x}_i)$ from the training set
- The so-called „support vectors“
- This type of SVM is called „**hard-margin SVM**“ and can be used if the classes are linearly separable Weil Klassen hart trennbar sind

Support Vector Machine (SVM)

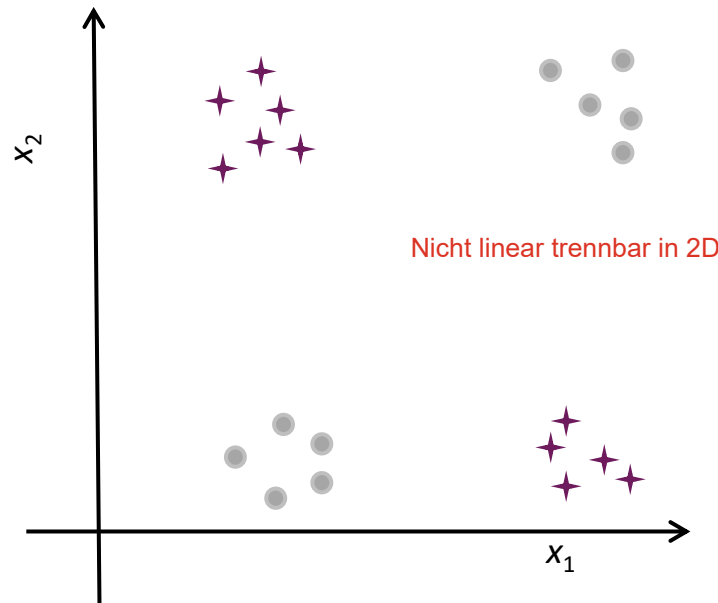


- A hard-margin SVM cannot find a decision function if the classes are not linearly separable.

Problem, da Inhalte nicht mehr hart trennbar sind.

- Solution:
 - allow some instances to be on the „wrong“ side of the supporting hyperplanes ()
 - penalize those instances by introducing so-called slack variables ξ_i : Wert groß --> bestrafen; Wert klein --> erlaubt man best. margin
 - $\xi_i > 0$ if the instance $\vec{x}_i$ is not within the boundary
 - $\xi_i = 0$ otherwise
- This type of SVM is called „**soft-margin SVM**“

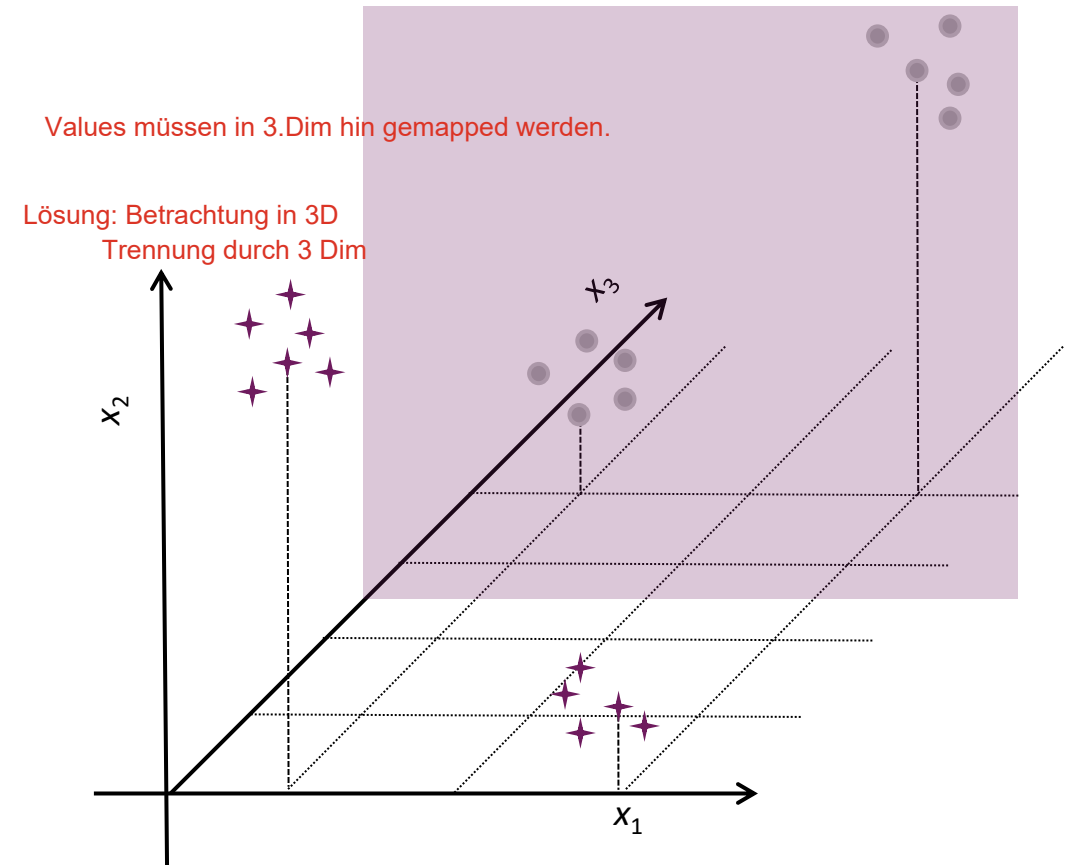
Support Vector Machine (SVM)



Nicht linear trennbar in 2D

$$R^2 \rightarrow R^3$$

● class ω_1
★ class ω_2



Values müssen in 3.Dim hin gemapped werden.

Lösung: Betrachtung in 3D
Trennung durch 3 Dim

If we....

...add a third dimension x_3 and „somehow“ shift the instances of class ω_1 on that dimension, we can linearly separate the two classes in the new feature space R^3

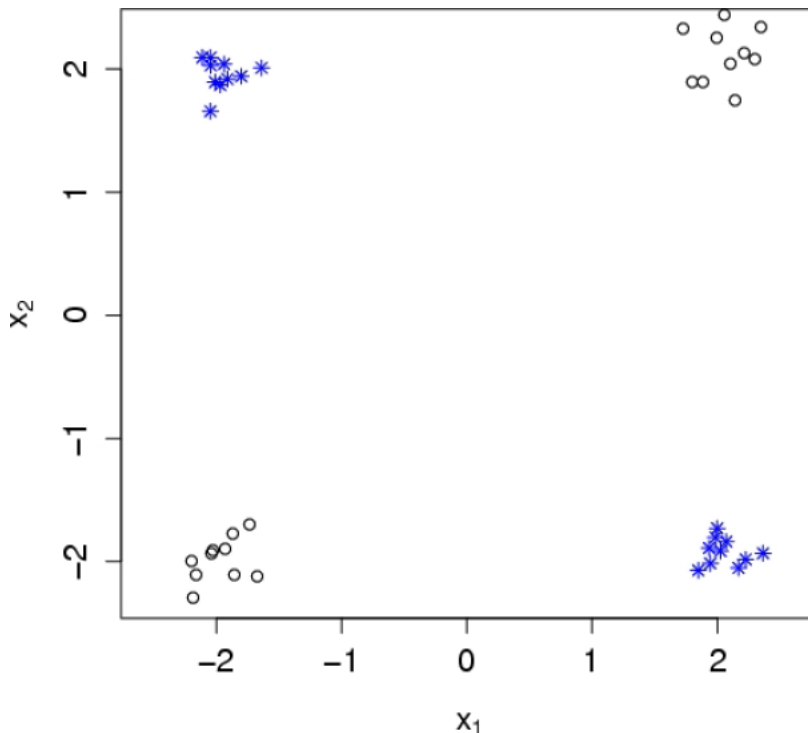
Support Vector Machines (SVM)

Mapping to higher-dimensional space using a mapping function

Let us consider a mapping function

$$Z := \phi(\vec{x})$$

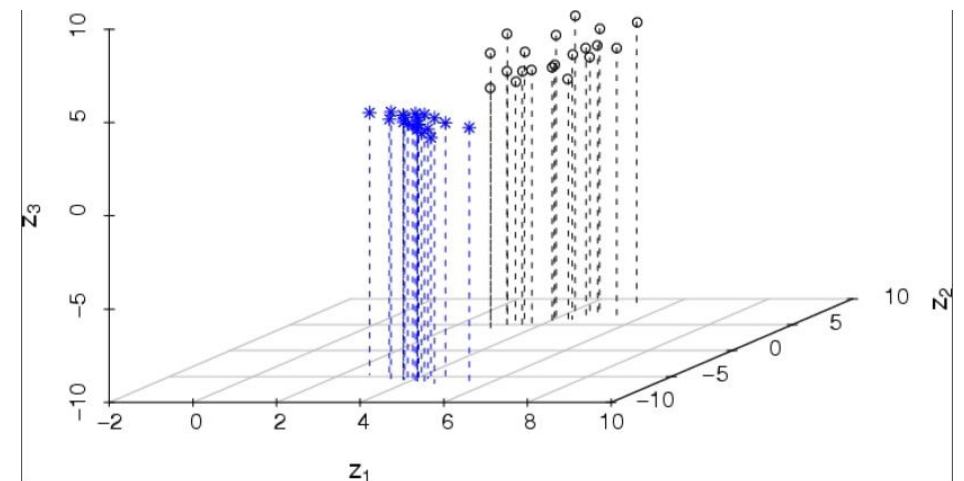
which maps $R^2 \rightarrow R^3$



Using the mapping

$$\begin{aligned} z_1 &:= x_1^2 \\ z_2 &:= \sqrt{2}x_1x_2 \\ z_3 &:= x_2^2 \end{aligned}$$

the classes can be linearly separated.



Support Vector Machine (SVM)

- For the mapping, we need to map each instance $\vec{x}_i$ to an instance $\vec{z}_i$ in the new feature space using $\vec{z}_i = \phi(\vec{x}_i)$.
- Incorporating $\phi()$ into the optimization problem yields

$$L(\vec{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \phi(\vec{x}_i) \phi(\vec{x}_j)$$

- **Problem:**
 - Obviously we cannot just use any mapping function. In the two previous examples, the mapping functions were ideal for the data set. They nicely separated the two classes. Mapping Funktion müssen wir hier noch definieren.
- If we need to find specific mapping functions for each data set, the approach would be infeasible.

Support Vector Machine (SVM)

- Looking at the optimization problem, we notice that $\vec{x}_i$ and $\vec{x}_j$ are represented by the inner product as $\phi(\vec{x}_i)\phi(\vec{x}_j)$
- Now we use the so-called „**kernel trick**“: Statt mapping function --> inner product austauschen --> aus inner product kommt 3 Dim
- Instead of actually doing the mapping using $\phi()$, we replace the inner product by
$$K(\vec{x}_i, \vec{x}_j) = \phi(\vec{x}_i)\phi(\vec{x}_j)$$
- which leads to

$$L(\vec{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j K(\vec{x}_i, \vec{x}_j)$$

- Now we use a **kernel function** $K(\vec{x}_i, \vec{x}_j)$ that returns a value for each pair of $\vec{x}_i$ and $\vec{x}_j$ has a parameter that can be tuned during training.

Support Vector Machine (SVM)

Two widely used kernel functions are

- polynomial kernel:

$$K(\vec{x}_i, \vec{x}_j) = (\vec{x}_i \vec{x}_j + r)^d$$

sometimes with scaling parameter γ : $K(\vec{x}_i, \vec{x}_j) = (\gamma \vec{x}_i \vec{x}_j + r)^d$

- radial basis function kernel (Gaussian): Standard

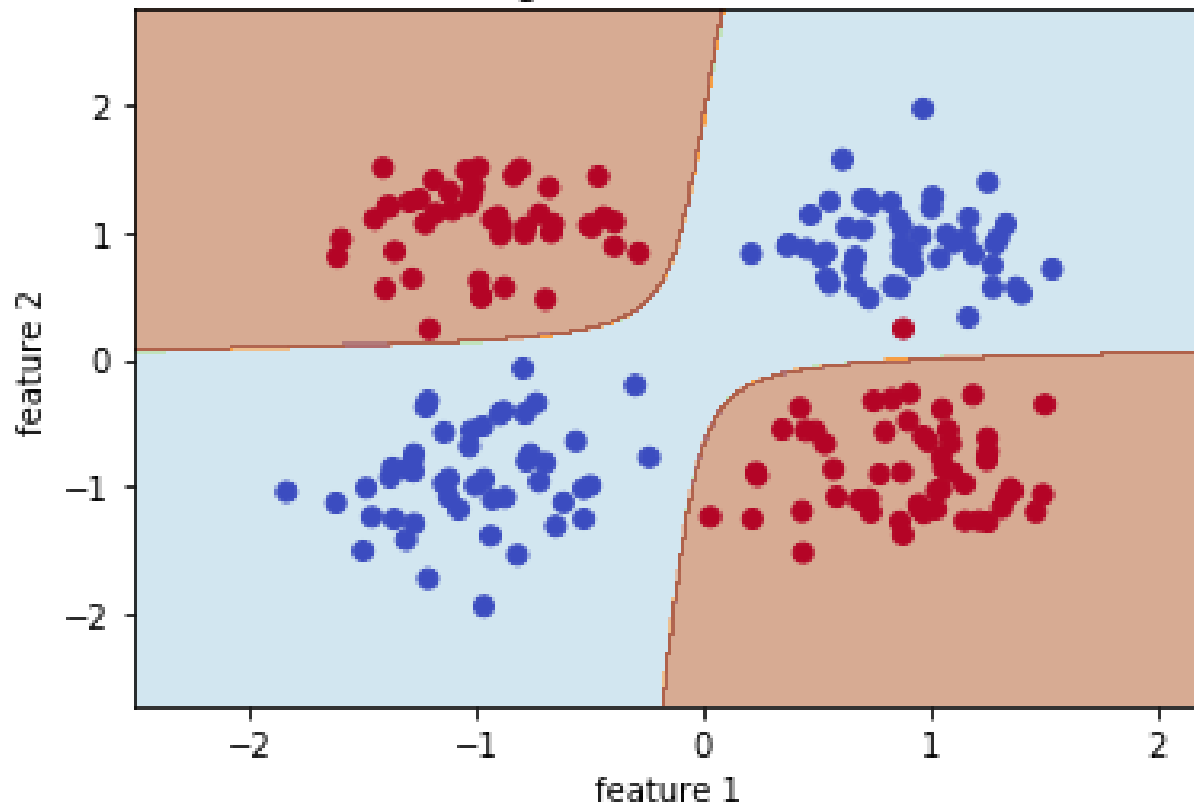
$$K(\vec{x}_i, \vec{x}_j) = e^{-\frac{\|\vec{x}_i - \vec{x}_j\|^2}{2\sigma^2}} = e^{-\gamma \|\vec{x}_i - \vec{x}_j\|^2} \quad \text{where } \gamma = \frac{1}{2\sigma^2}$$

- For non-linear problems one typically starts with the radial basis function (RBF) kernel.

Support Vector Machine (SVM)

Durch 3 Dim wird diese Linien linear

soft-margin SVM with RBF-kernel



- Usually soft-margin SVMs with non-linear kernels like the RBF kernel are used.
- During training the parameter C and the kernel parameter (σ for the RBF kernel) are tuned so that the training data is separated with a low error rate. Großes C bestraft stark
- Looking at classification results in the original feature space, it can be seen that this type of SVM finds non-linear decision functions.

Support Vector Machine (SVM)

```
# create dataset
```

```
X, y = make_moons(n_samples=500, noise=0.1, random_state=123)
```

```
# split into training and test data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
    test_size=0.5)
```

```
# fit scaler on training data only
```

```
scaler = MinMaxScaler().fit(X_train)
```

```
# scale training and test data
```

```
X_train = scaler.transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# Soft-margin SVM with RBF kernel
```

```
# SVC tries to determine a good value for gamma, if gamma = 'auto'
```

```
model = SVC(kernel = 'rbf', gamma = 'auto', C = 1)
```

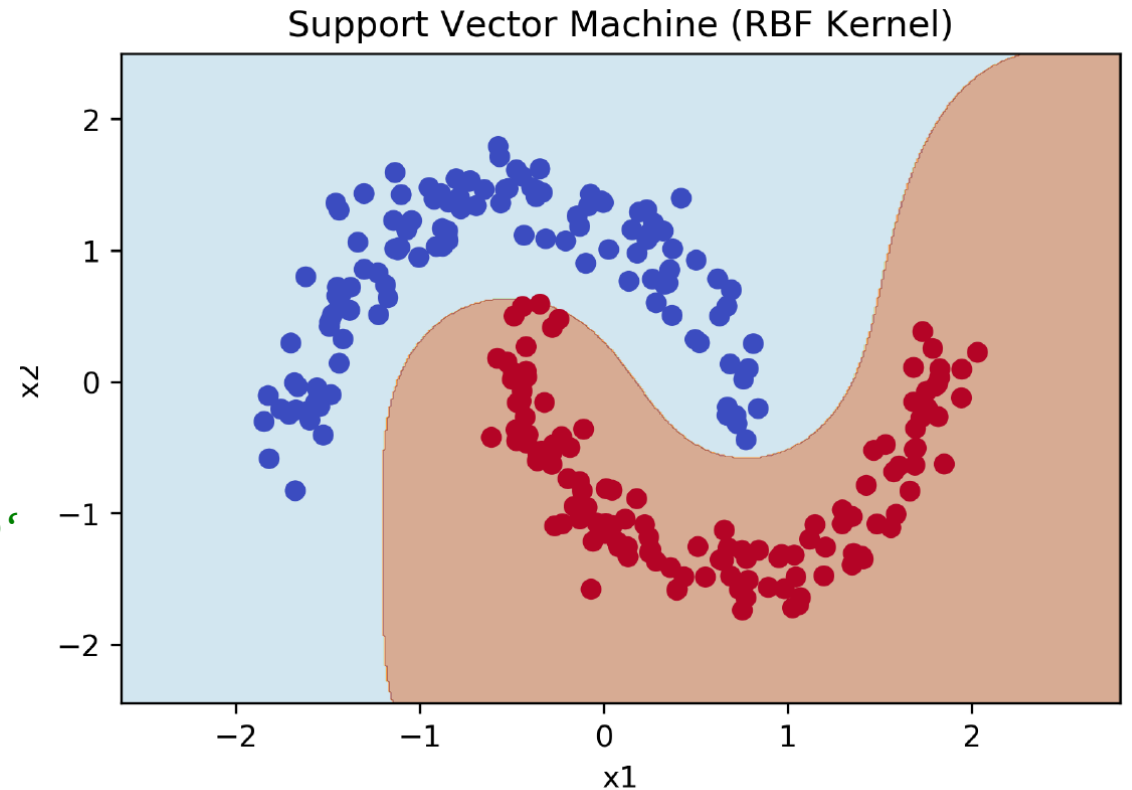
```
model.fit(X_train, y_train)
```

```
# predict labels for test data
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print(confusion_matrix(y_test, y_pred))
```



Support Vector Machine (SVM)

- Advantage
 - robustness: robust against noise in the training set if soft-margin SVMs are used
 - highly-flexible decision boundary if kernel trick is used
 - global optimum is found (for a given set of hyperparameters)
 - no random parts involved, in contrast to ANNs
- Disadvantage
 - in the standard case works only for numeric data
 - (data transformation or specific kernels required)
 - interpretability: poor interpretability, if the data is mapped to higher-dimensional space

Key-Takeaways

- The goal of classification is to assign instances to predefined classes.
- Linear classifiers such as Logistic Regression work well when classes are linearly separable.
- Non-linear classifiers are needed when decision boundaries are more complex, for example in the XOR problem. *Was wäre Problem bei non-lin-Problem? (Prüfung)*
- Different classifiers have different strengths:
 - Naive Bayes: simple and interpretable, but based on strong independence assumptions
 - Logistic Regression: fast, interpretable, and a strong baseline
 - k-NN: intuitive and easy to implement, but sensitive to scaling and computationally expensive at prediction time
 - Random Forest: robust and powerful, but less interpretable
 - XGBoost: often very strong predictive performance, but more complex
 - SVM: effective for complex decision boundaries, especially with kernels
- There is no universally best classifier, the right choice depends on the data, the task, interpretability requirements, and computational cost. *Kommt auf Datensatz an.*

Control Questions

- Why can a Random Forest often outperform a single Decision Tree?
- Why can Naive Bayes still work well in practice even though its independence assumption is often unrealistic?
- Why is hyperparameter tuning especially important for models such as SVM, Random Forest, or XGBoost?
- In which situation would you choose Logistic Regression instead of XGBoost, and why?
- Why can k-NN be attractive for small datasets but problematic for large datasets?
- Why is the Bayes Error theoretically important for understanding classification problems?

References

- Chen, Q., Cao, F., Xing, Y., & Liang, J. (2023). Evaluating Classification Model Against Bayes Error Rate. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 45(08), 9639–9653.
- Friedman, Jerome. (2000). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*. 29.
- Chen, Tianqi and Guestrin, Carlos (2016), XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD 2016*